

Requirement Engineering



Introduction to requirement gathering

- ? Requirements are one of the most vital pieces to ensuring the success of a system or project
- ? To ensure the optimal requirements are received, the methods in which those requirements are obtained are equally important.
- ? The second phase of the systems development life cycle is analysis phase

What is requirement gathering?

- ? Requirement gathering is the act of generating a list of requirements to define what a project is about and its goal.
- ? Poorly established requirements can have a negative impact, while properly established ones can lead to success
- ? Requirements are the “blueprints” that everyone involved on the project uses to work from.

Requirement gathering techniques

? Each requirement gathering technique has advantages. Cost and time are important factors when picking which method to use and many times you can use more than one to ensure you gather all the relevant information needed.

1. Open ended Questionnaire

? Open-ended questions are questions that allow someone to give a free-form answer.

? The most important benefit of open-ended questions is that they **allow you to find more than you anticipate**: people may share motivations that you didn't expect and mention behaviors and concerns that you knew nothing about.

? Start open questions with "how" or with words that begin with "w," such as "what," "when," "where," "which," and "who."

? **When to Ask Open-Ended Questions**

- ? In a screening questionnaire, when recruiting participants for a usability study (for example, "How often do you shop online?")
- ? While conducting design research, such as on
 - ? Which problems to solve
 - ? What kind of solution to provide
 - ? Who to design for
- ? For exploratory studies, such as
 - ? Qualitative usability testing
 - ? Interviews and other field studies
 - ? Diary studies
 - ? Personal research
 - ? Use-case research
 - ? Task analysis

2. Closed-ended questions

- ? Closed-ended questions can be answered with “Yes” or “No,” or they have a limited set of possible answers (such as: A, B, C, or All of the Above).
- ? Closed-ended questions are often good for **surveys**, because you get higher response rates when users don’t have to type so much. Also, answers to closed-ended questions can easily be analyzed statistically, which is what you usually want to do with survey data.
- ? **When To Ask Closed-Ended Questions**
- ? In *quantitative* usability studies, where you are measuring time on task and error rates, and you need to compare results among users
- ? In surveys where you expect many (1000+) respondents
- ? When collecting data that must be measured carefully over time, for example with repeated (identical) research efforts
- ? When the set of possible answers is strictly limited for some reason
- ? After you have done enough qualitative research that you have excellent multiple-choice questions that cover most of the cases

3.Survey

- ? Offering a survey or questionnaire allows you to collect information from many people in a relatively short amount of time, particularly helpful for interacting with people in different geographic locations and also good for budget savings and time constraints. When preparing your survey, consider these tips:
- ? Keep them shorter versus longer so people are more likely to complete them
- ? Focus on a feature or topic, rather than many at once
- ? Use ratings to generate data analysis responses, like "*strongly agree*," "*agree*" or "*disagree*"
- ? Have some open-ended questions to allow free-form responses to get detailed input
- ? Use the six question words to structure your survey: who, what, when, where, why and how, like "*How does the user login*," or "*Where are the results shown in the program?*"

Joint Application Development (JAD)

- ▶ Also known as Joint Application Design
- ▶ Used for requirements elicitation
- ▶ Group orientated
- ▶ Facilitated through workshop sessions



4 Joint Application Design/JAD:

- ? Joint Application Development (JAD) was introduced in the late 1970s
- ? Joint application design (JAD) often speeds up the construction of information systems by combining group dynamics and customer involvement to come up with a solution jointly.
- ? JAD is successful in reducing time because the user and other key members are heavily involved in the process. The goal is to get the design right the first time, thereby reducing different iterations
- ? Joint Application Development (JAD) is a process used to collect business requirements while developing new information systems for a company. The JAD process may also include approaches for enhancing user participation, expediting development and improving the quality of specifications.



JAD

- ? **Joint Application Design (JAD)** is a **structured, collaborative requirements-gathering technique** in which **users, business stakeholders, and IT teams work together in facilitated workshops** to quickly define, analyze, and agree on system requirements.
- ? Instead of conducting many separate interviews and meetings, JAD brings **all key participants into one session** to make decisions collectively.

Facilitator Responsibilities

- ▶ Is a neutral party
- ▶ Preparation for JAD Workshops
- ▶ Facilitate Workshop sessions
- ▶ Follow Agenda
- ▶ Keep team focused
- ▶ Resolve conflicts



Participant Responsibilities

- ▶ Adhere to Workshop Rules and procedures
- ▶ Attend all workshop sessions
- ▶ Actively participate in sessions
- ▶ Discuss high-level requirements
- ▶ Propose requirements and specifications
- ▶ Work towards consensus on decisions



Scribe Responsibilities

- ▶ Documentation
 - ▶ Record agreed upon requirements and specifications
 - ▶ Record all open issues
 - ▶ Gather created documents and materials
 - ▶ Write the requirements document



Preparing for JAD Workshop

- ▶ Establish rules and procedures
- ▶ Interview participants
- ▶ Gather materials
- ▶ Establish meeting agenda



The JAD Workshop

- ▶ Facilitator follows agenda
- ▶ Discuss high-level requirements
- ▶ Decisions through group consensus
- ▶ Unresolved open issues are documented
- ▶ Design specification is created





Upside

- ▶ JAD allows stakeholders to rapidly decide on solutions for various problems.
- ▶ JAD allows all stakeholders to speak directly with one another.
- ▶ JAD provides a structured approach to requirement elicitation that can help reduce development errors.



Downside

- ▶ JAD requires the team to have substantial experience in the problem domain.
- ▶ JAD offers quick solutions to problems, so often these problems are considered too quickly.
- ▶ Members of large meetings can become subject to “group think.”
- ▶ JAD can be logistically difficult if there are a large number of stakeholders spread across different time zones.

The benefits of a JAD meeting include:

- ? Simplifying the process and saving time by consolidating phone calls, emails and other meetings
- ? Identifying participants, users and issues quickly and collectively
- ? Clarifying the requirements unanimously
- ? Transitioning from one phase of development to the next seamlessly
- ? Satisfying the customer because they helped develop the system and approved the stages of work

Types of requirements:

Functional Requirements

Functional requirements are product features or functions that developers must implement to enable users to accomplish their tasks. So it's essential to make them clear both for the development team and the stakeholders. Generally, functional requirements describe system behavior under specific conditions.

Functional requirements examples

Functional requirements will vary for different types of software. For example, functional requirements for a website or mobile application should define user flows and various interaction scenarios.

The system sends a confirmation email when a new user account is created.

The system sends an approval request after the user enters personal information.

A search feature allows users to search content/items by entering the query in the search bar.

The user can review items in the cart, change their number, or remove them before checkout.

The app should allow users to create accounts and log in using credentials like email and password or through social media integration.

The app can send notifications to users for updates, reminders, or promotional content.

Users should be able to provide feedback or rate services/products within the app.

These are some common functional requirements. More specialized software systems will have more specific requirements. For example, a [hotel property management system](#) will include such requirements as “the user should be able to view and update room status” or “the system must aggregate bills from all [points of service](#) in a folio.”

Nonfunctional requirements

Nonfunctional requirements are not related to the system's functionality but rather define *how* the system should perform. They are crucial for ensuring the system's usability, reliability, and efficiency, often influencing the overall user experience..

Nonfunctional requirements examples

Some examples of nonfunctional requirements are:

The website pages should load in 3 seconds with the total number of simultaneous users <5 thousand.

The system should be able to handle 20 million users without performance deterioration.

The payment processing gateway must be PCI DSS compliant.

A program running on Windows 10 must be able to run on Windows 11 without any change in its behavior and performance.

FUNCTIONAL vs NONFUNCTIONAL REQUIREMENTS

	Functional requirements	Nonfunctional requirements
Objective	Describe what the product does	Describe how the product works
End result	Define product features	Define product properties
Focus	Focus on user requirements	Focus on user expectations
Essentiality	They are mandatory	They are not mandatory but desirable
Origin type	Usually defined by the user	Usually defined by developers or other tech experts
Testing	Component, API, UI testing, etc. Tested before nonfunctional testing	Performance, usability, security testing, etc. Tested after functional testing
Types	Authentication, authorization levels, data processing, reporting, etc.	Usability, reliability, scalability, performance, etc.

Business requirements

These include high-level statements of goals, objectives, and needs. Business requirements do not have any details or specific features. They just state the problem and the business objective to be achieved, such as

- increased revenue/throughput/customer reach,
- reduced expenses/errors,
- improved customer service, etc.

User requirements

This group of requirements reflects the needs of discrete stakeholder groups (top-level managers, nonmanagement staff, customers, etc.) and defines what they expect from a particular solution. They serve as a bridge between generalized business requirements and specific solution requirements. They are outlined in a User Requirements Specification and can include, for example, the ability to create various reports, view [order](#) history and status, manage customer databases, etc.

Requirement Engineering

The broad spectrum of tasks and techniques that lead to an understanding of requirements is called requirements engineering. From a software process perspective, requirements engineering is a major software engineering action that begins during the communication activity and continues into the modeling activity.

Requirement Engineering

- ? Requirement Engineering is the process of defining, documenting and maintaining the requirements.
- ? Requirements engineering encompasses seven distinct tasks: **inception, elicitation, elaboration, negotiation, specification, validation, and management.**
- ? Requirements engineering begins with inception. It moves onward to elicitation, and then elaboration.

Inception

- ? How does a software project get started?
- ? The basic aim is to understand the problem
- ? To Know who will use Software
- ? To know exact nature of problem that is expected from customers side
- ? What will be economical benefits of a successful solution.

Requirement Elicitation

- ? Requirements elicitation (also called requirements gathering) combines elements of problem solving, elaboration, negotiation, and specification.
- ? An important part of elicitation is to establish business goals.
- ? Once the goals have been captured, a prioritization mechanism should be established, and a design rationale for a potential architecture can be created.

Elaboration

- ? The information obtained from the customer during inception and elicitation is expanded and refined during elaboration.
- ? Elaboration is driven by the creation and refinement of user scenarios that describe how the end user (and other actors) will interact with the system.
- ? Each user scenario is parsed to extract analysis classes—business domain entities that are visible to the end user.

Negotiation.

- ? Discussion on financial and other Commercial issue.
- ? In this Customer , user , stakeholders discuss to decode:
- ? To rank the Requirement
- ? To decide priorities
- ? To decide risks
- ? To finalize the product cost

Specification.

- ? A specification can be a written document, a set of graphical models, a formal mathematical model, a collection of usage scenarios, a prototype, or any combination of these.
- ? For large systems, a written document, combining natural language descriptions and graphical models may be the best approach.

Validation.

- ? Requirements validation examines the specification to ensure that all software requirements have been stated unambiguously; that inconsistencies, omissions, and errors have been detected and corrected; and that the work products conform to the standards established for the process, the project, and the product.
- ? The primary requirements validation mechanism is the technical review

Requirements management.

- ? Requirements for computer-based systems change and the desire to change requirements persists throughout the life of the system.
- ? Requirements management is a set of activities that help the project team identify, control, and track requirements and changes to requirements at any time as the project proceeds. Many of these activities are identical to the software configuration management (SCM) techniques

Software Requirement Specification (SRS)

- ? SRS is a work product that is created when a detailed description of all aspects of the software to be built must be specified before the project is to commence.
- ? It include a set of use cases that describes all interactions that the user will have with the software.
- ? It also contains functional as well as non functional requirements.

Software Requirement Specification (SRS)

- ? First, the SRS could be written by the client of a system in which they define the needs and expectation.
- ? Second, the SRS could be written by a developer of the system. SRS is written for various purposes and serves as a contract document between customer and developer.
- ? Every requirement in the SRS should be identifiable by a unique number or tag.

Need of SRS

- ? A contract between the customer and the software vendor
- ? Enables costing and pricing of the project
- ? Input for detailed design
- ? Management of customer expectations

A good SRS document usually contains project scope section, functional requirements, requirement analysis models, external interface requirements and non functional requirements.

SRS - Scope of the project/ Product vision

- ? It include a brief overview of the project and should also indicate the goals of the project including its benefits. Sometimes it is better to separate the project scope into a separate document.

SRS - Functional Requirements

- ? Functional requirements specify the business requirements of the project in detail.
- ? It should contain a combination of use cases and plain textual description of system features.
- ? To ensure that all the business requirements are addressed in the final software product, a traceability matrix document is used. Traceability matrix tracks each requirement through various phases of software development.

SRS - Requirement Analysis Models

- Requirement Analysis models act as the bridge between functional requirements and the detailed design of the software system.
- For example, Use cases lead to user interface design, data dictionary and entity relationship diagrams are used for designing database schema and class diagrams.

SRS – Non-Functional Requirements

- Non functional or technical requirements specify how the software system should operate.
- Non functional requirements captured include performance requirements, application scalability, application security, maintainability, usability, availability, logging and auditing, data migration requirements, multi lingual support etc.

Use Cases

As requirements are gathered, an overall vision of system functions and features begin to materialize. To understand how these functions and features will be used by different classes of end users, developers and users can create a set of scenarios that identify a thread of usage for the system to be constructed. The scenarios, often called use cases , provide a description of how the system will be used.

DEVELOPING USE CASES

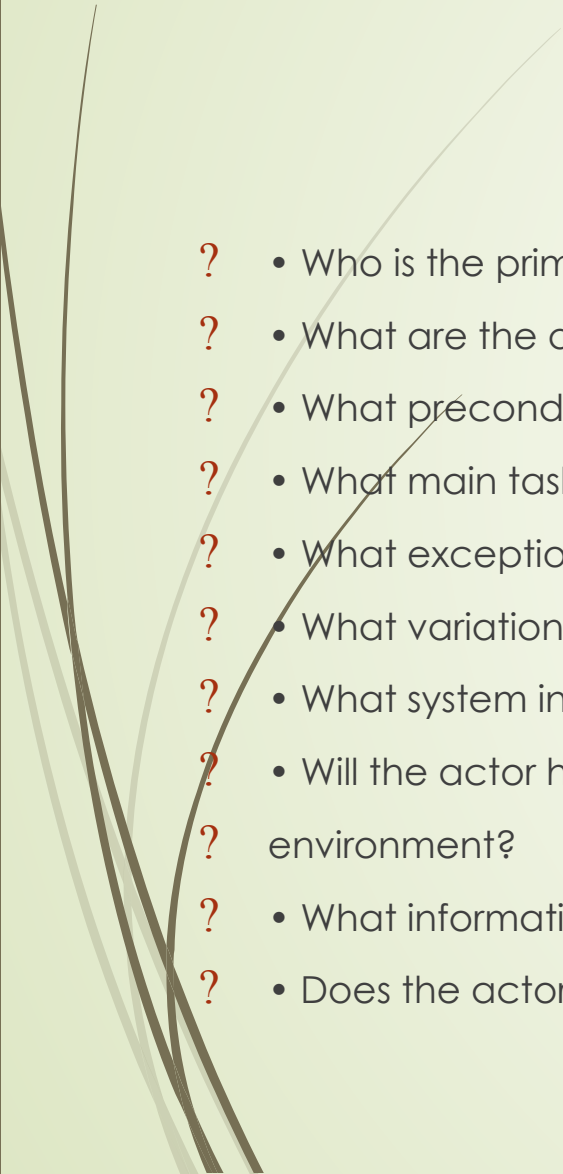
A use case tells a story about how an end user (playing one of a number of possible roles) interacts with the system under a specific set of circumstances. The story may be narrative text, an outline of tasks or interactions, a template-based description, or a diagrammatic representation. Regardless of its form, a use case depicts the software or system from the end user's point of view.

Steps for writing Use Case

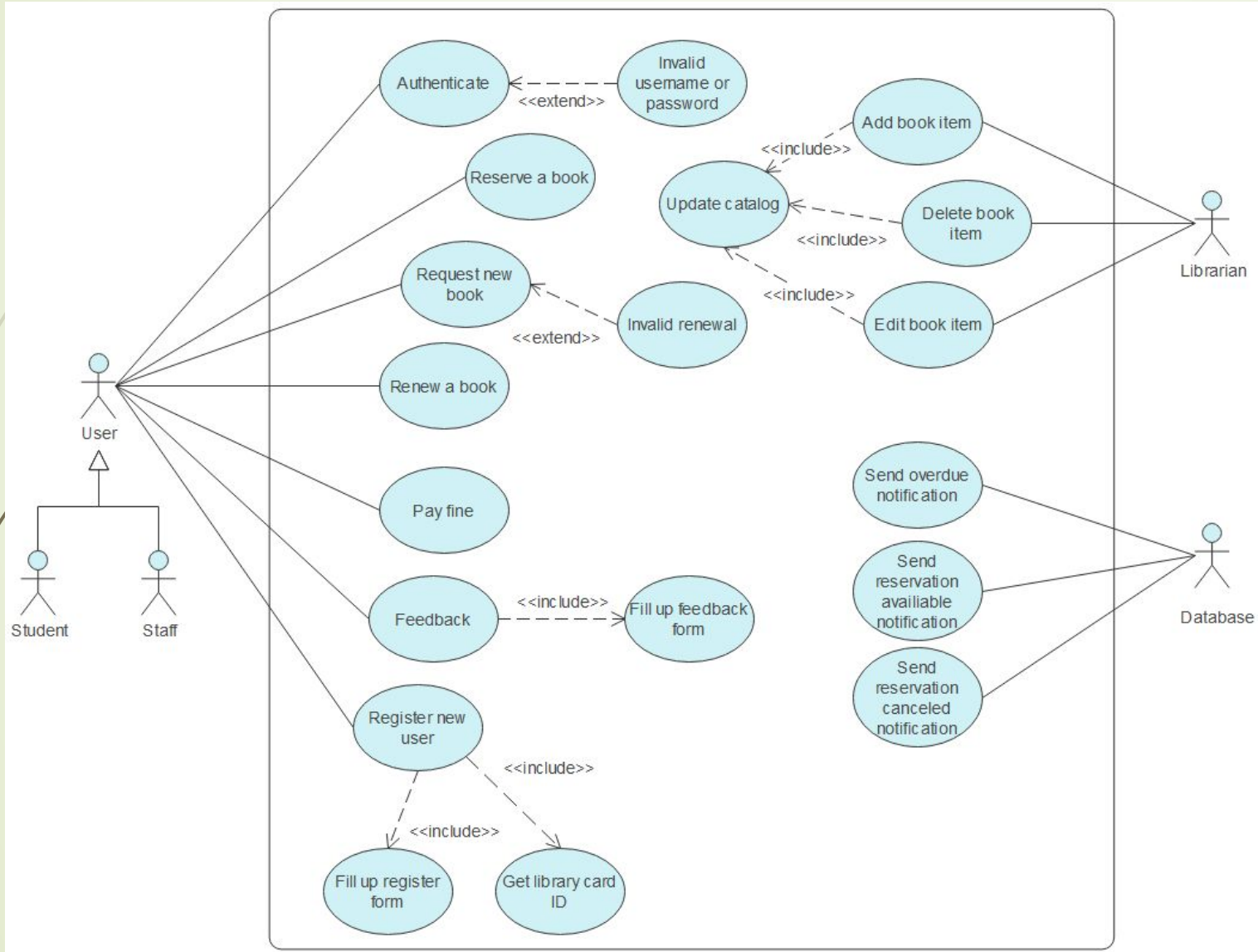
- ? Define the set of “actors” that will be involved in the story. Actors are the different people (or devices) that use the system or product within the context of the function and behaviour that is to be described.
- ? Actors represent the roles that people (or devices) play as the system operates.
- ? An actor is anything that communicates with the system or product and that is external to the system itself.

- 
- ? Primary actors interact to achieve required system function and derive the intended benefit from the system. They work directly and frequently with the software.
 - ? Secondary actors support the system so that primary actors can do their work.

Use cases

- 
- ? • Who is the primary actor, the secondary actor(s)?
 - ? • What are the actor's goals?
 - ? • What preconditions should exist before the story begins?
 - ? • What main tasks or functions are performed by the actor?
 - ? • What exceptions might be considered as the story is described?
 - ? • What variations in the actor's interaction are possible?
 - ? • What system information will the actor acquire, produce, or change?
 - ? • Will the actor have to inform the system about changes in the external environment?
 - ? • What information does the actor desire from the system?
 - ? • Does the actor wish to be informed about unexpected changes?

Use Case Diagram for Library Management System



UML – Unified Modelling Language

- UML is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems.
- UML is not a programming language but tools can be used to generate code in various languages using UML diagrams.
- UML diagrams are not only made for developers but also for business users, common people, and anybody interested to understand the system.

features of UML

- It is a generalized modeling language.
- It is distinct from other programming languages like C++, Python, etc.
- It is interrelated to object-oriented analysis and design.
- It is used to visualize the workflow of the system.
- It is a pictorial language, used to generate powerful modeling artifacts.

UML categories

There are two broad categories of diagrams and they are again divided into subcategories –

- ? Structural Diagrams
- ? Behavioral Diagrams

I. Structural Diagrams

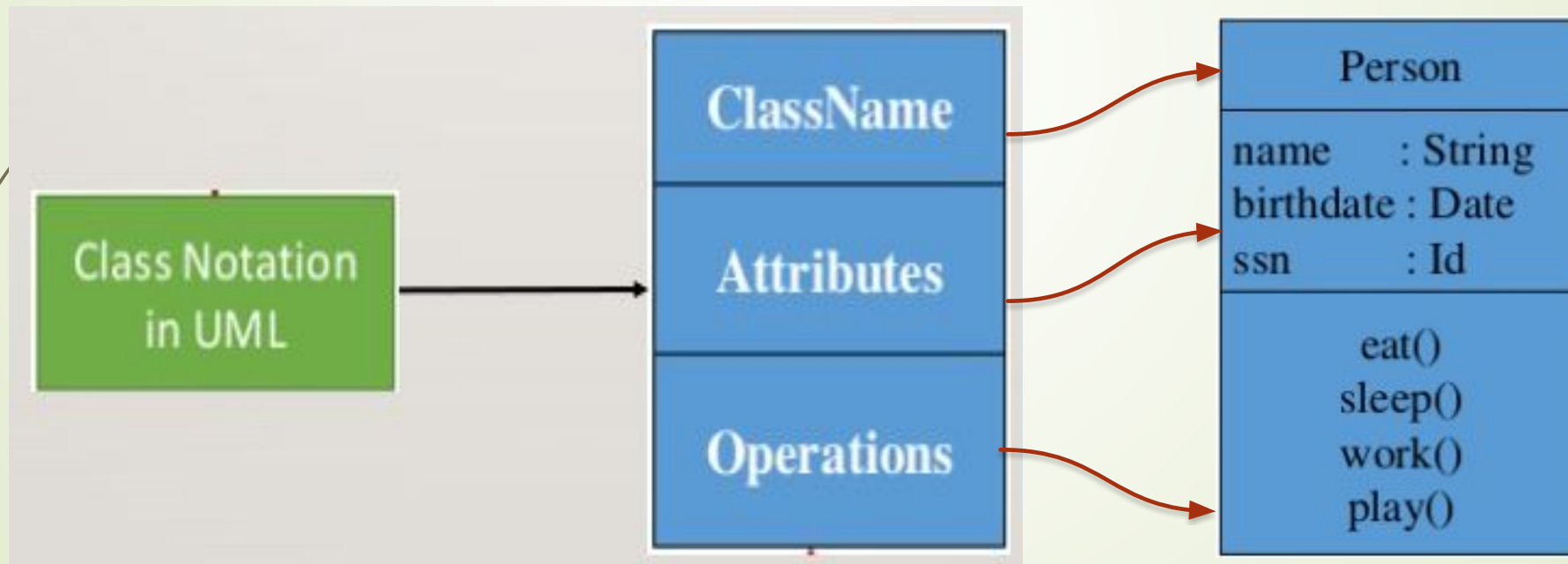
- **Class Diagram**
- **Data Flow Diagrams**

II. Behavioral Diagrams

- **Use case diagram**
- **Sequence diagram**

1. Class Diagram

It describes the structure of a system by showing the system's classes, its attributes, operations (or methods) and relationships among objects.



1. Class Diagram

- Class attributes can be listed in the form of:

attributeName : Type

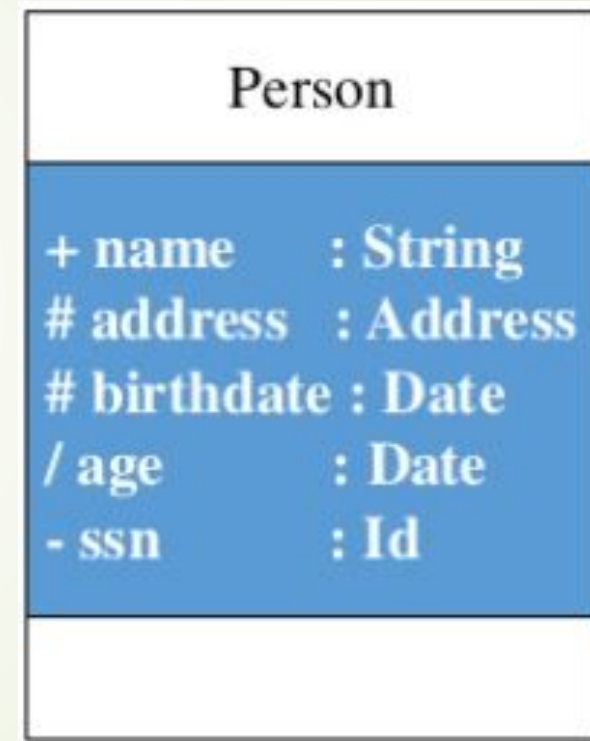
- Attributes can be :

- ? public : denoted by “ + ”

- ? Protected : denoted by “ # ”

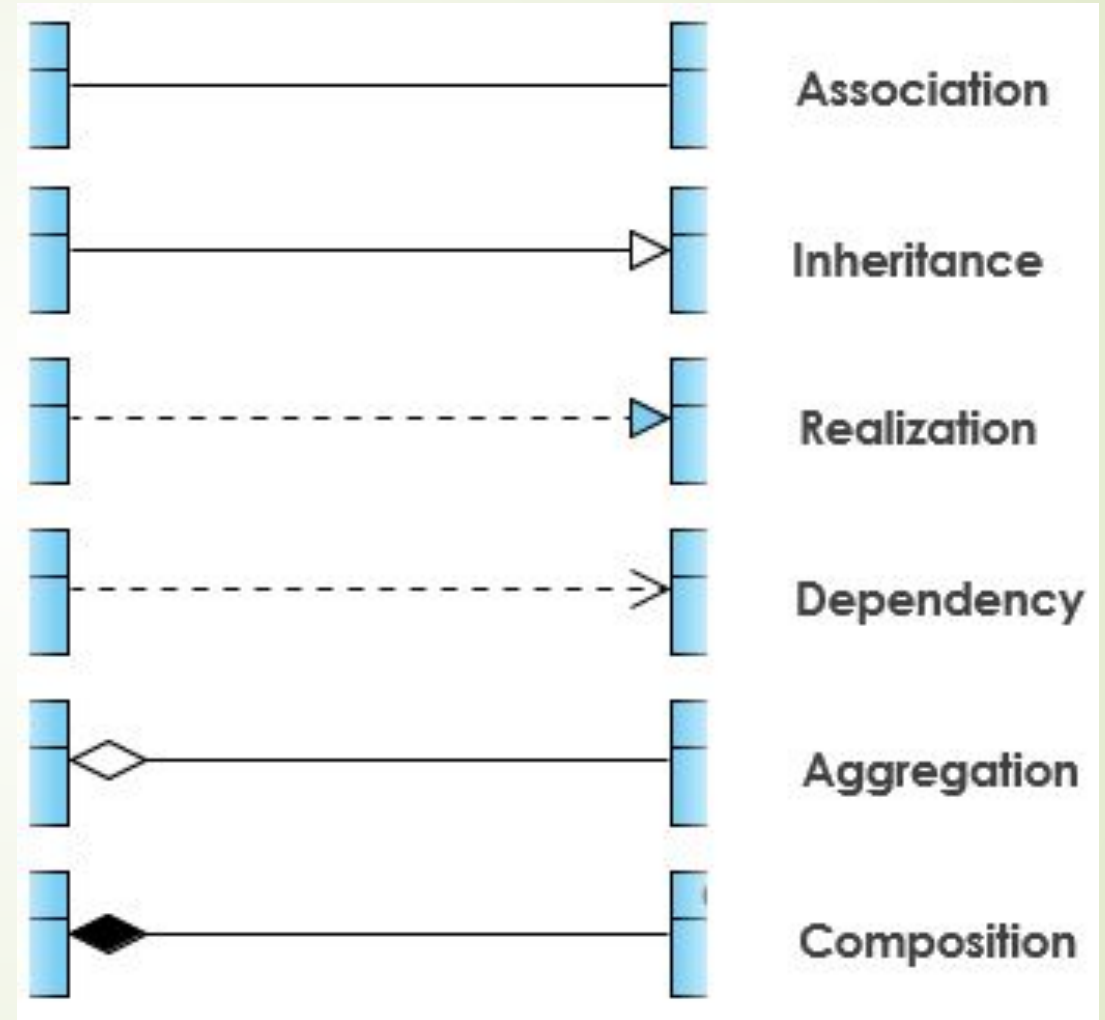
- ? Private : denoted by “ - ”

- ? derived : denoted by “ / ”



1. Class Diagram- Relationship Between Classes

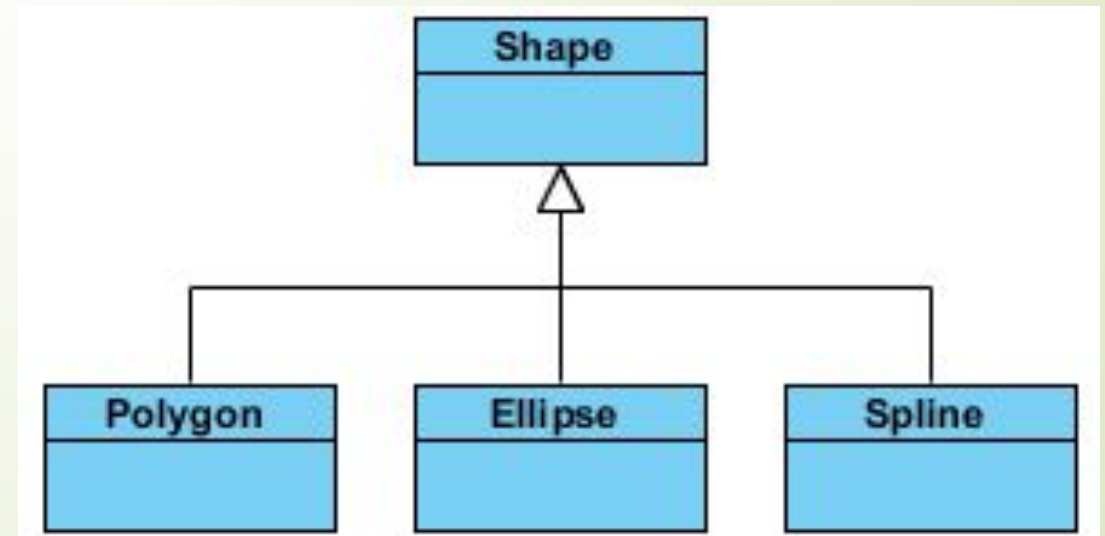
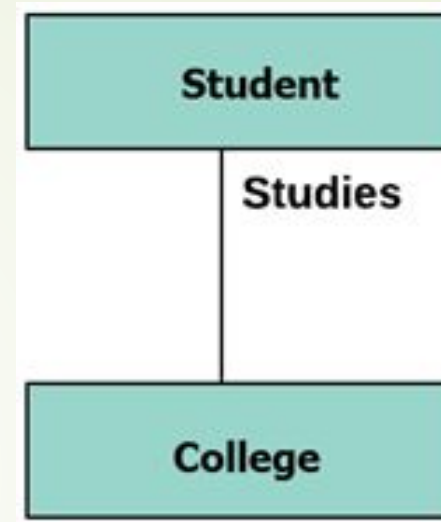
A class may be involved in one or more relationships with other classes. A relationship can be one of the following types:



1. Class Diagram- Relationship Between Classes

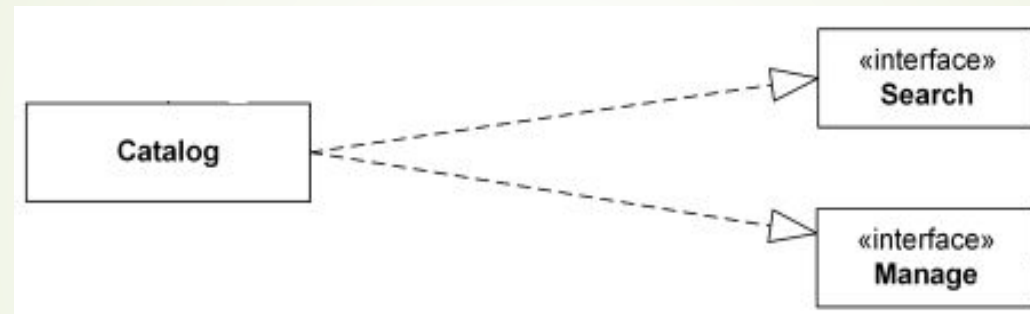
? **Association:** It represents static relationships between classes A and B. For example; the relationship between student and college is shown which is studies.

? **Inheritance:** It represent parent child relationship.



1. Class Diagram- Relationship Between Classes

? **Realization:** It represents the relationship between the interface and the implementing class. For example, *Library Catalog* provides access for students to search books and also allow staff to manage library items

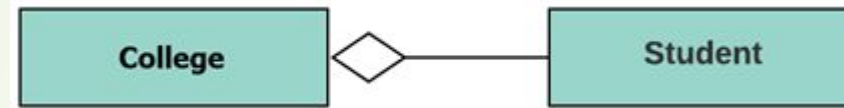


? **Dependency:** Dependency indicates that one class depends on another. For example, Student has a dependency on College.



1. Class Diagram- Relationship Between Classes

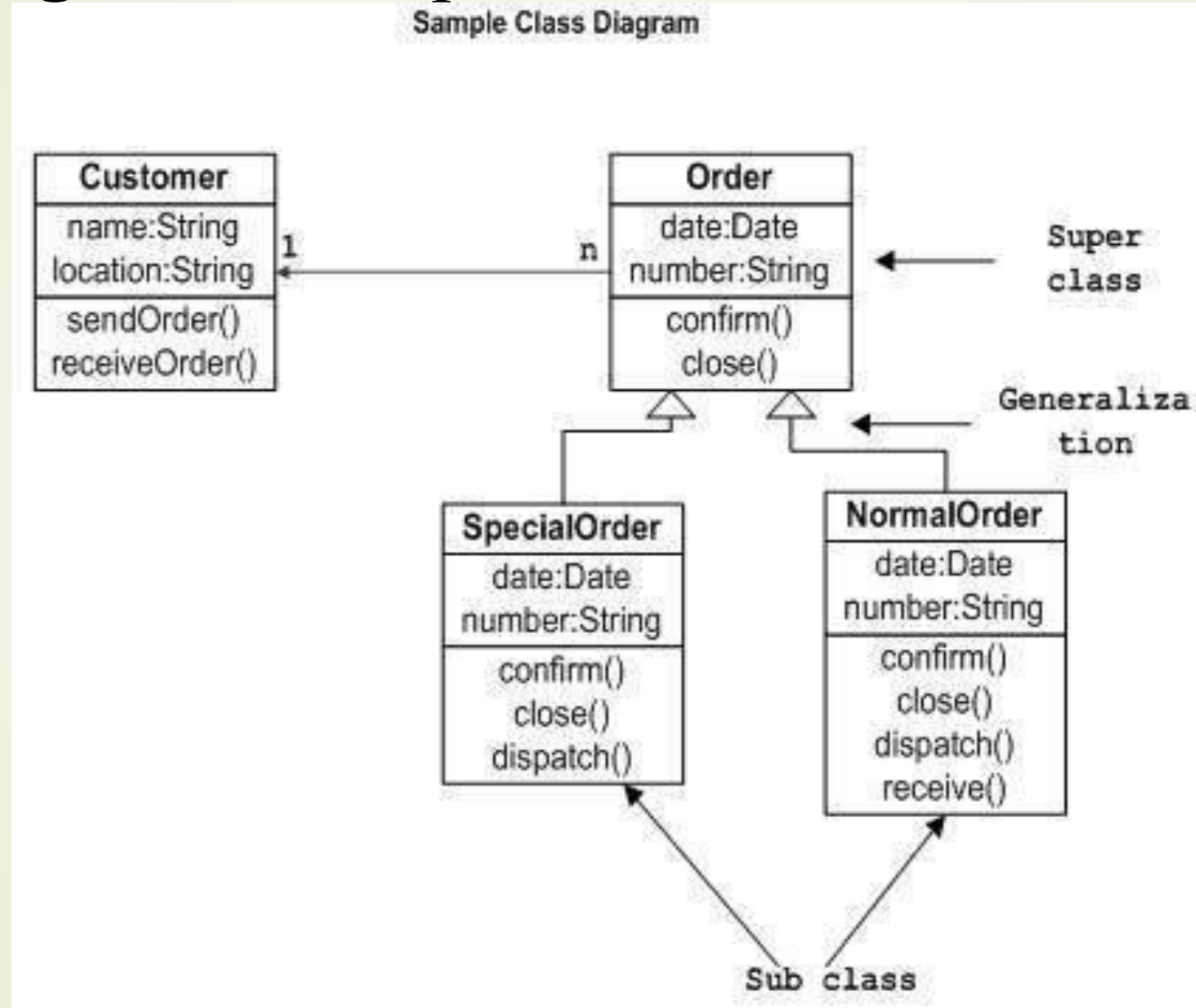
? **Aggregation:** In this, the contained classes are never totally dependent on the lifecycle of the container. Here, the college class will remain even if the student is not available.



? **Composition:** It denotes strong ownership between two classes when one class is a part of another class. For example, if college is composed of classes student. The college could contain many students, while each student belongs to only one college. So, if college is not functioning all the students also removed.



1. Class Diagram- Example



Object Diagram

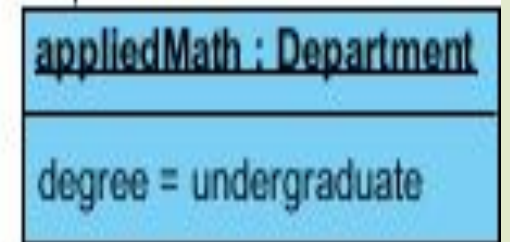
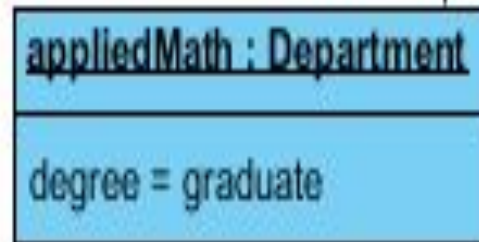
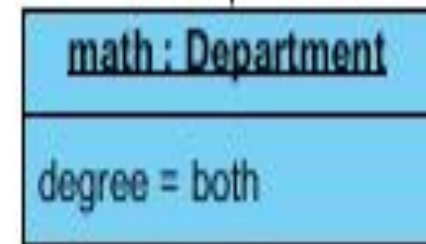
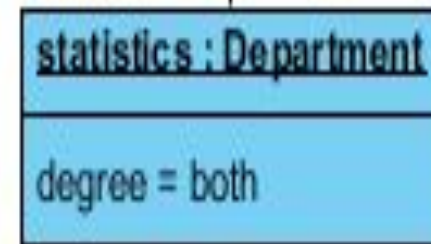
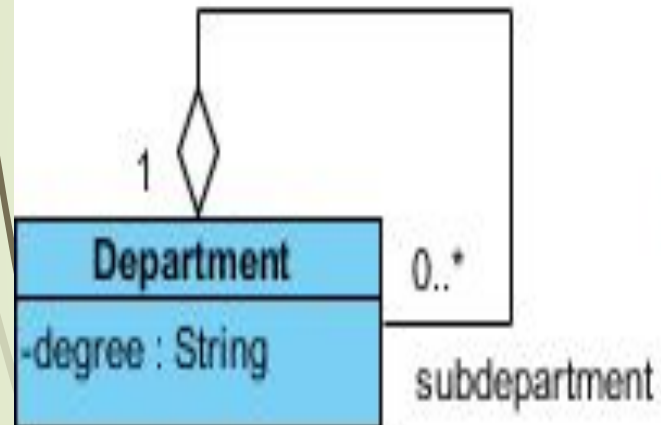
- ? Object is an instance of a class in a particular moment in runtime that can have its own state and data values.
- ? Likewise a static UML object diagram is an instance of a class diagram;
- ? it shows a snapshot of the detailed state of a system at a point in time, thus an object diagram encompasses objects and their relationships which may be considered a special case of a class diagram or a communication diagram.

Purpose of Object Diagram

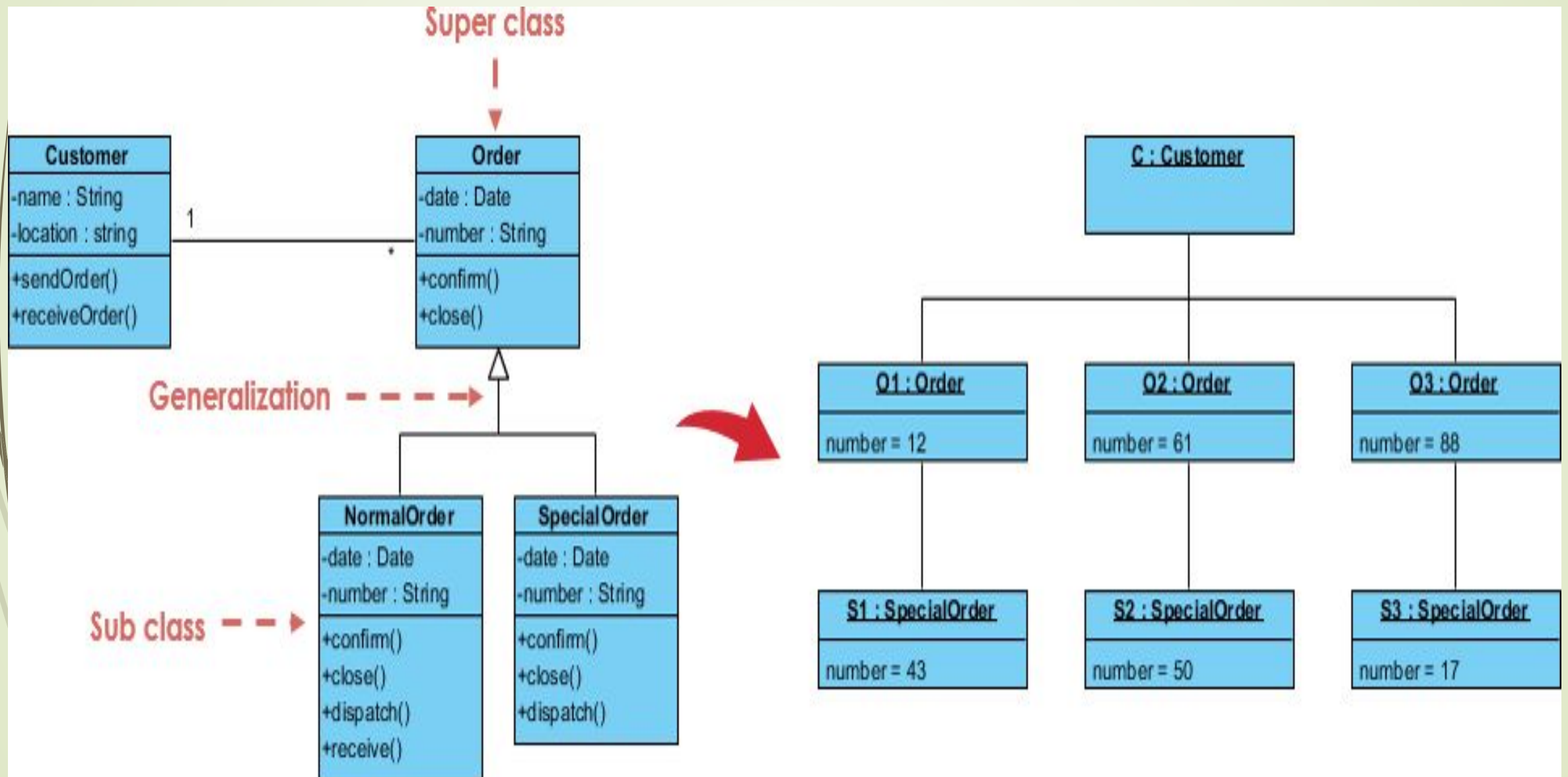
The use of object diagrams is fairly limited, mainly to show examples of data structures.

- During the analysis phase of a project, you might create a class diagram to describe the structure of a system and then create a set of object diagrams as test cases to verify the accuracy and completeness of the class diagram.
- Before you create a class diagram, you might create an object diagram to discover facts about specific model elements and their links, or to illustrate specific examples of the classifiers that are required.

Instance name -- --> mathStat : Department <-- -- Class name



Class to Object diagram example



Component Diagram

It describes all the individual components that are used to make the functionalities, but not the functionalities of the system. It visualizes the physical components inside the system. The components can be a library, packages, files, etc.

The component diagram also describes the static view of a system, which includes the organization of components at a particular instant. The collection of component diagrams represents a whole system.

The main purpose of the component diagram are enlisted below:

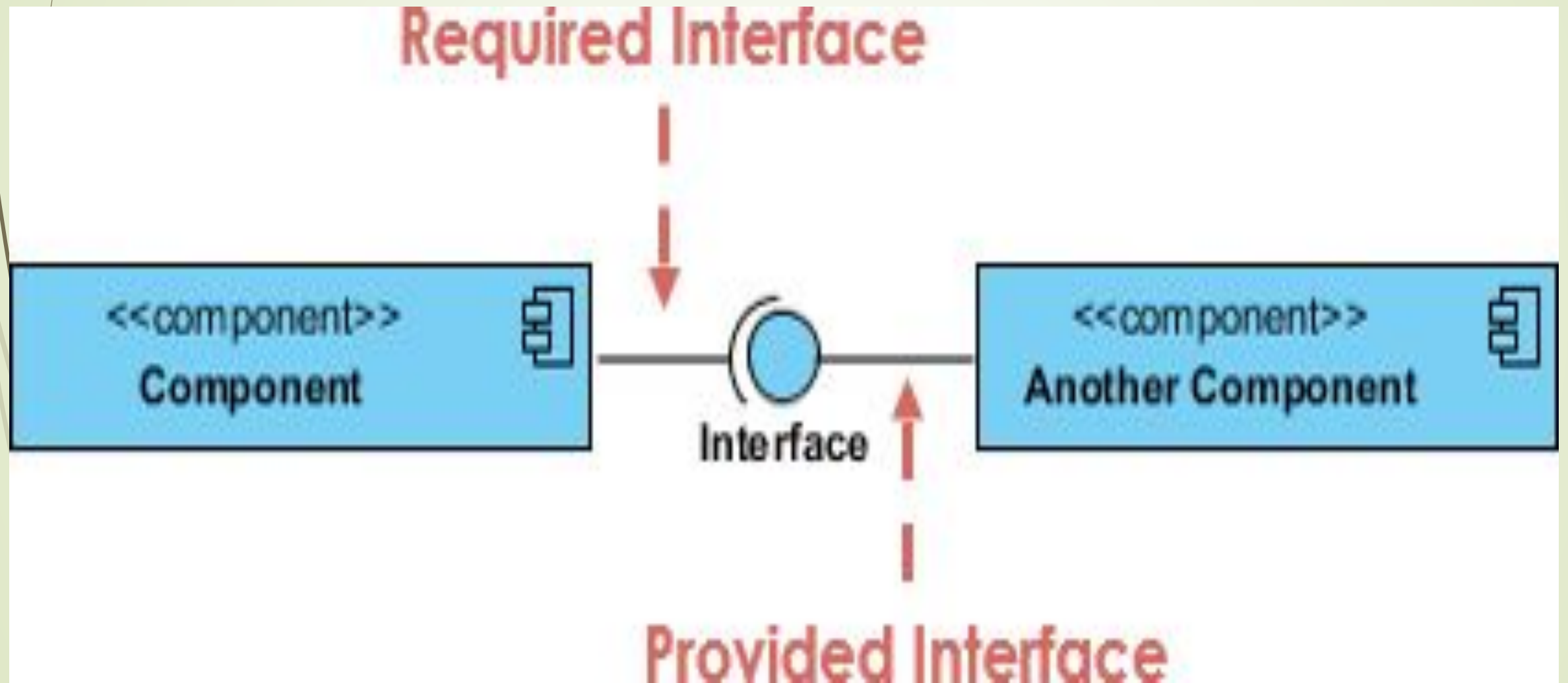
1. It envisions each component of a system.
2. It constructs the executable by incorporating forward and reverse engineering.
3. It depicts the relationships and organization of components.

Basic Concepts of Component Diagram

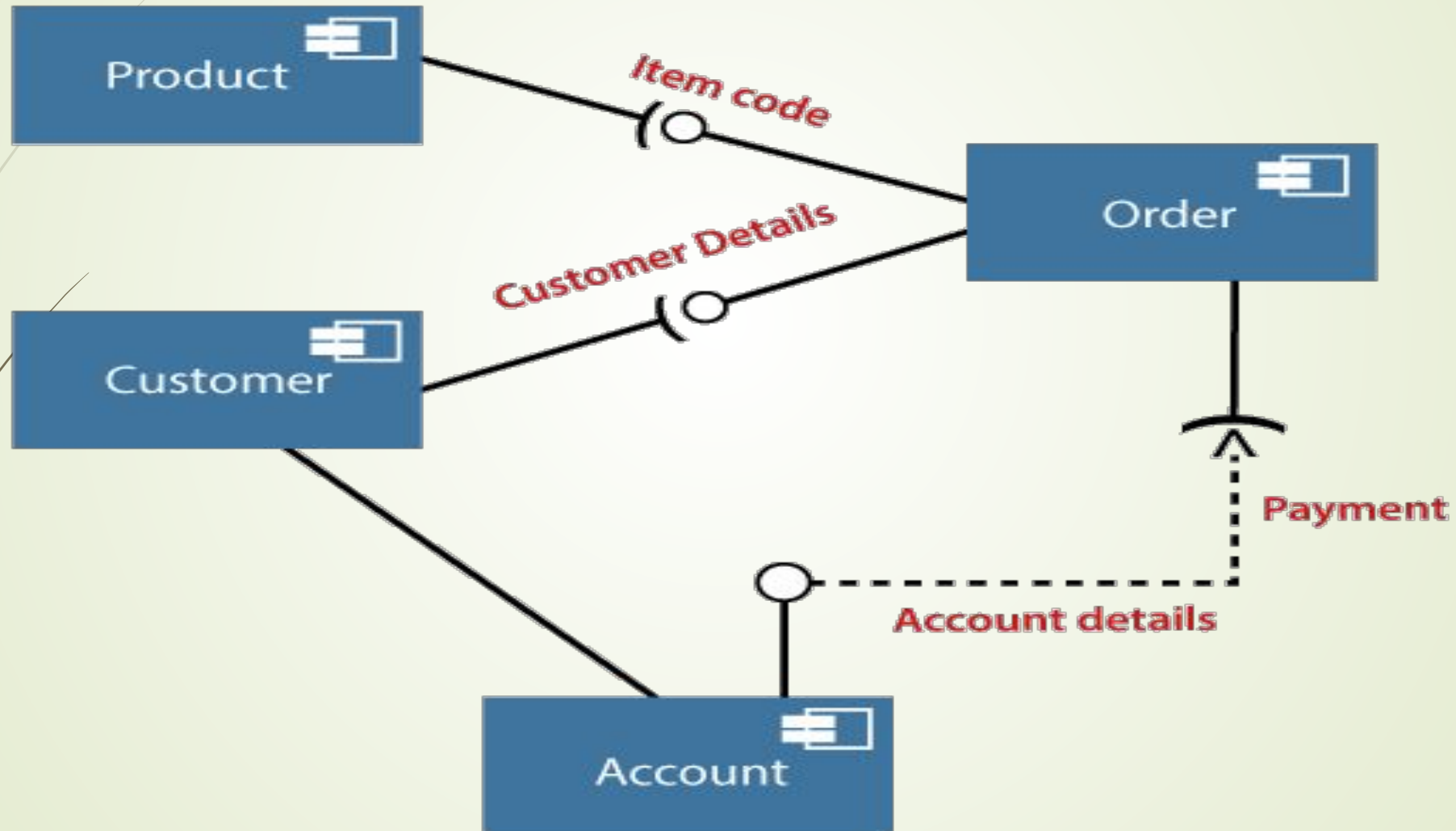
1. A rectangle with the component's name
2. A rectangle with the component icon
3. A rectangle with the stereotype text and/or icon



Interface



Component Diagram for online shopping system



DFD(Data Flow Diagram)

- Developing Data Flow Diagrams(DFD)
 - a) What are DFDs?
 - b) Symbols used in DFD
 - c) Rules of data flow
 - d) Good style in drawing DFD
- Describing systems with DFD & Levelling DFDs
- Logical & Physical DFDs

WHY DFD ?

Provides an overview of

- What data a system processes
- What transformations are performed
- What data are stored
- What results are produced and where they flow

WHY DFD ?

- Graphical nature makes it a good communication tool between
 - User and analyst
 - Analyst and System designer
- Structure of DFD allows starting from a broad overview and expand it to a hierarchy of detailed diagrams

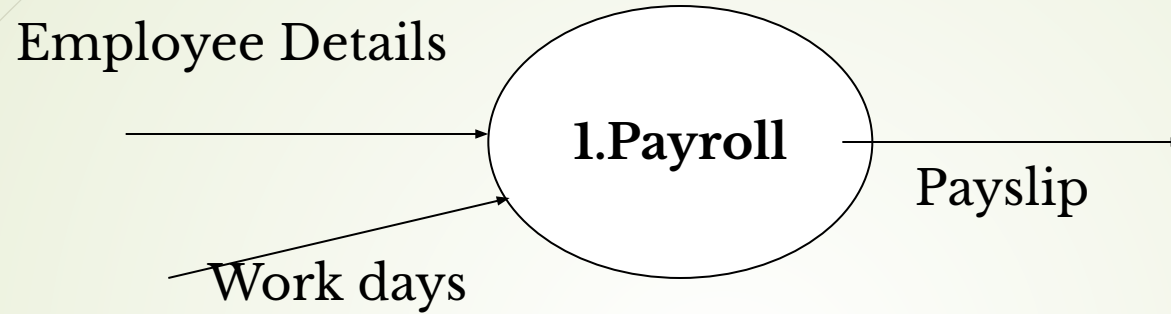
WHAT ARE DATA FLOW DIAGRAMS (DFD's)

DFDs models the system by depicting

- External entities from which the data flows and where results terminate
- Processes which transform data flows
- Data stores from which the data are read or into which data are written by the processes.

SYMBOLS USED IN DFD

PROCESS



- A circle represents a process
- Straight lines with incoming arrows are input data flows
- Straight lines with outgoing arrows are output data flows
- Processes are given serial numbers for easy reference
- Labels are assigned to Data flow. These aid documentation

SYMBOLS USED IN DFD

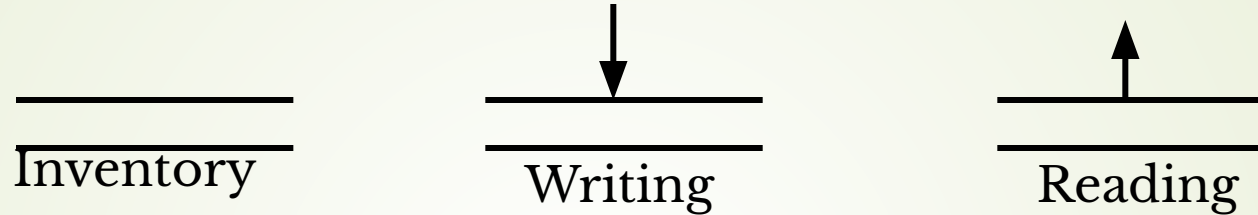
EXTERNAL ENTITIES



- A **Rectangle** represents an external entity
- They either supply data or receive data
- They **do not process data**

SYMBOLS USED IN DFD

DATA STORES



- A Data Store is a repository of data
- Data can be **written into the data store**

This is depicted by an incoming arrow

- Data can be **read from a data store**

This is depicted by an outgoing arrow

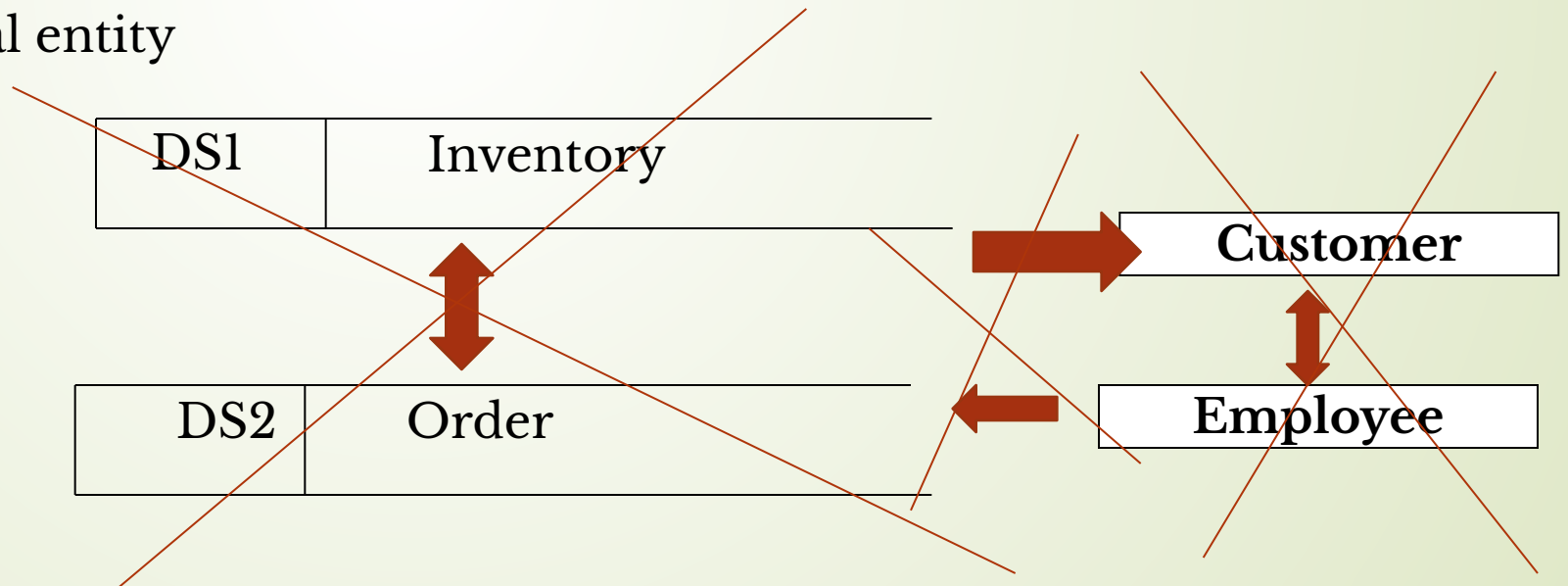
- **External entity** cannot read or write to the data store
- **Two data stores** cannot be connected by a data flow

RULES OF DATA FLOW

- Data can flow from
 - external entity to process
 - process to external entity
 - process to store and back
 - process to process
- Data cannot flow from
 - external entity to external entity
 - external entity to store
 - store to external entity
 - store to store

A Data store

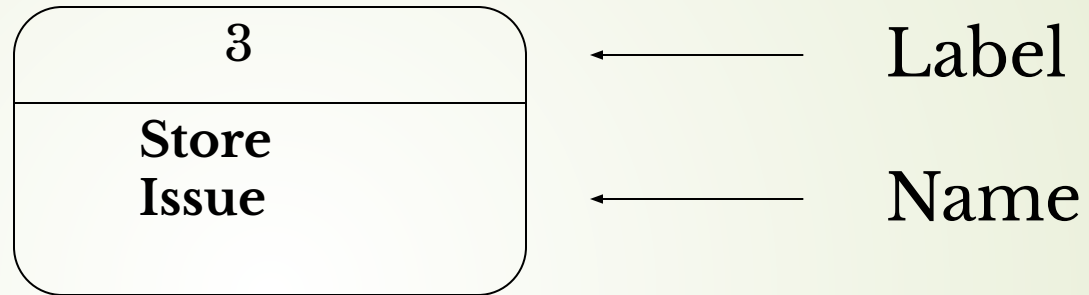
A Data store



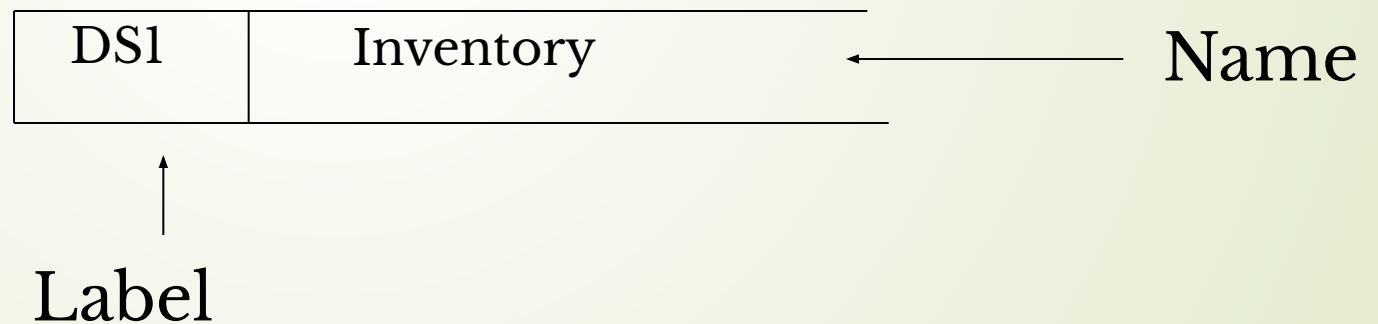
DATA FLOW DIAGRAMS

An alternate notation is often used

A Process



A Data store



GOOD STYLE IN DRAWING DFD

- Use **meaningful names** for data flows, processes and data stores.
- Use **top down development** starting from **context diagram** and successively levelling DFD
- Only **previously stored data can be read**
- A process can **only transfer input to output** .It cannot create new data
- **Data stores** cannot create new data

Creating Data Flow Diagrams

Steps:

1. Create a **list of activities**
2. Construct **Context Level DFD**
(identifies external entities and processes)
3. **Construct Level 0 DFD**
(identifies manageable sub process)
4. **Construct Level 1- n DFD**
(identifies actual data flows and data stores)
5. **Check against rules of DFD**

DFD Naming Guidelines

- External Entity □ Noun
- Data Flow □ Names of data
- Process □ verb phrase
 - a system name
 - a subsystem name
- Data Store □ Noun

LEVELLING DFD

- A context diagram **gives an overview**
- It should be **split into major processes** which give **greater detail**.
- Each **major process** is further **split to give more detail**.
- Each major process is further split to give more detail

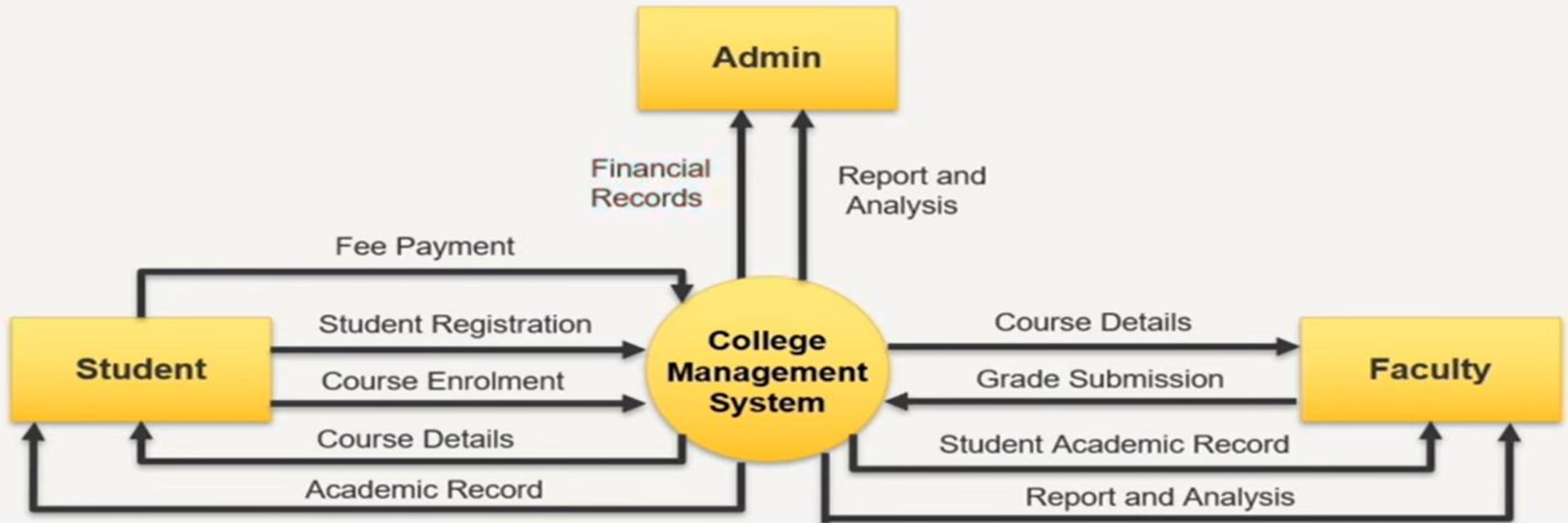
WHY LEVEL DFD?

- If a DFD is **too detailed** it will have **too many data flows** and will be large and difficult to understand
- **Start from a broad overview.** Expand to details - Idea similar to using procedures and linking these with a main program
- Each DFD must deal with **one aspect of a big system**

Level 0 DFD -Context Diagram

College Management System – Level 0

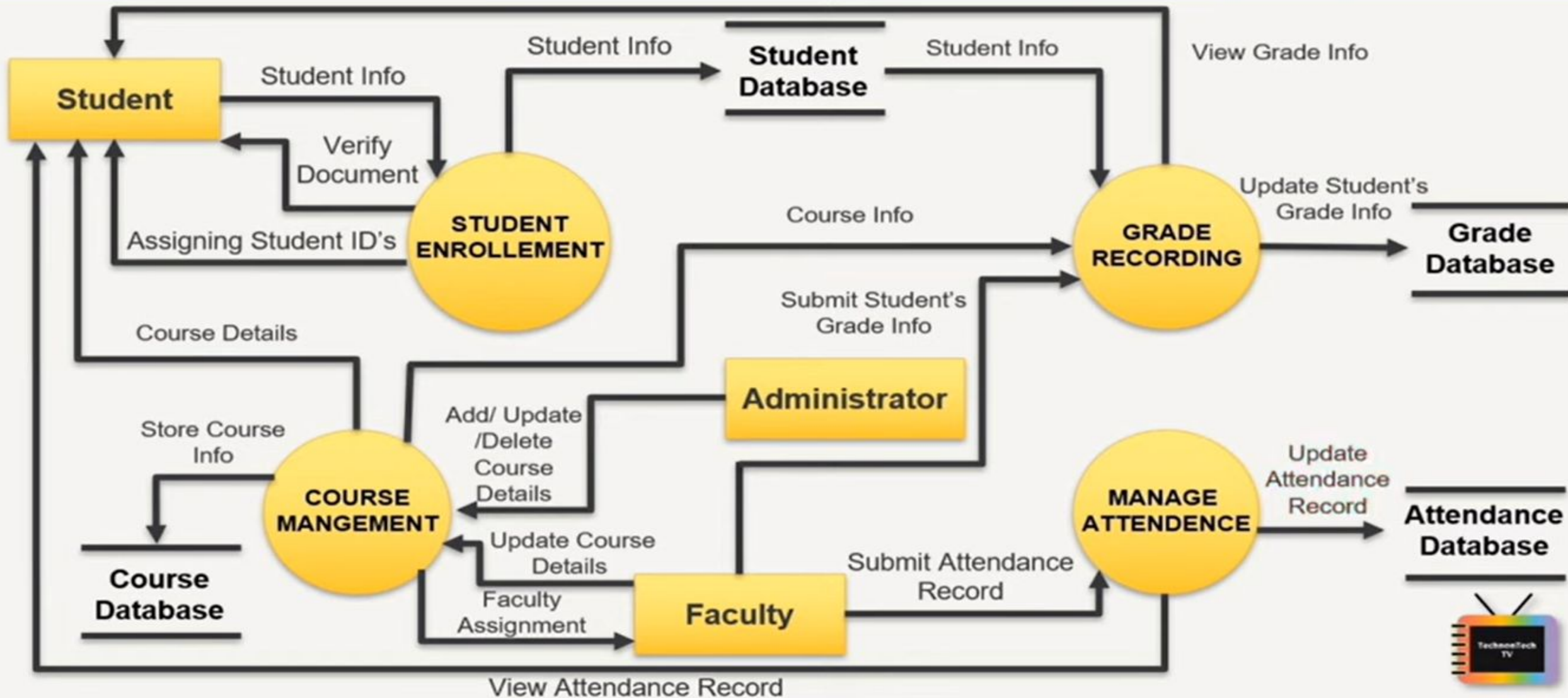
DFD Diagram



Level 1 DFD

College Management System – Level 1

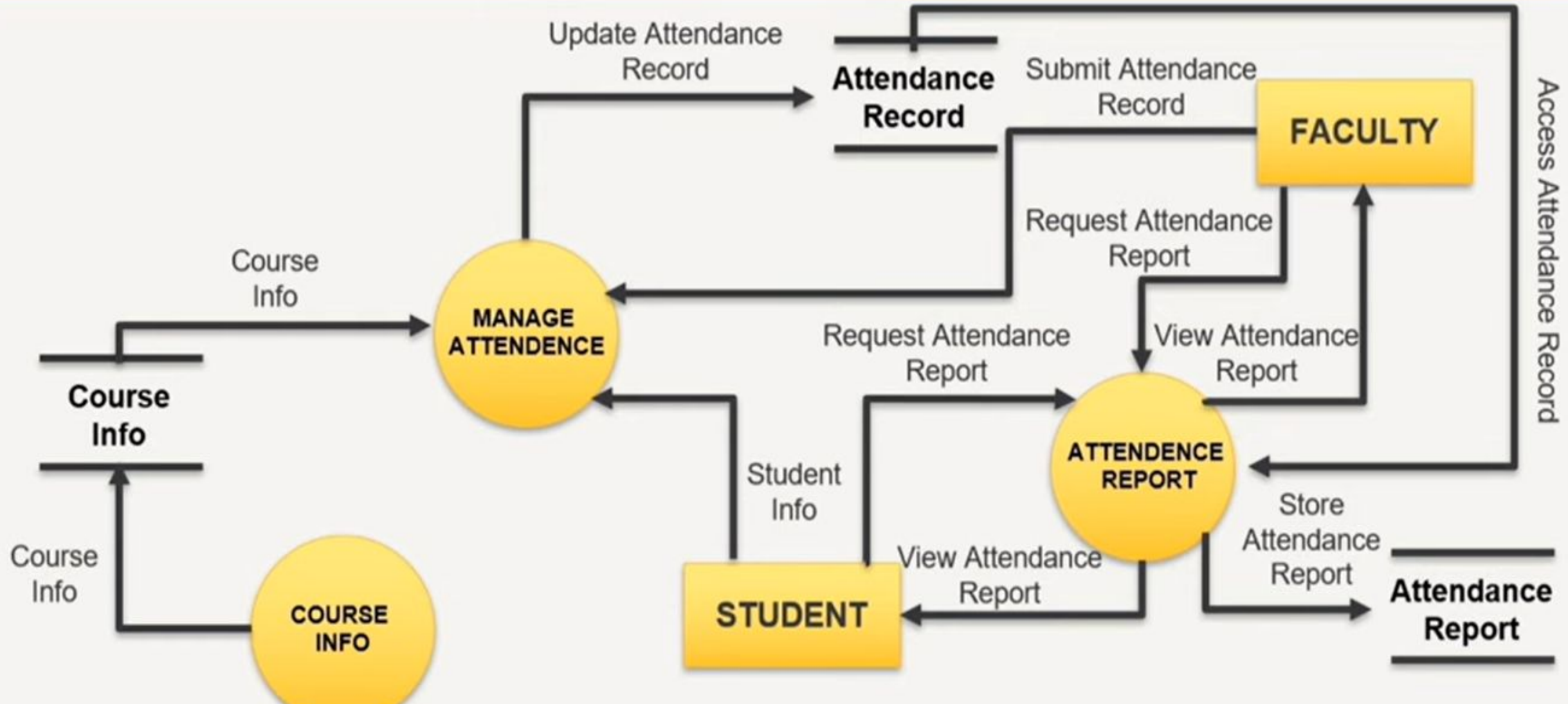
DFD Diagram



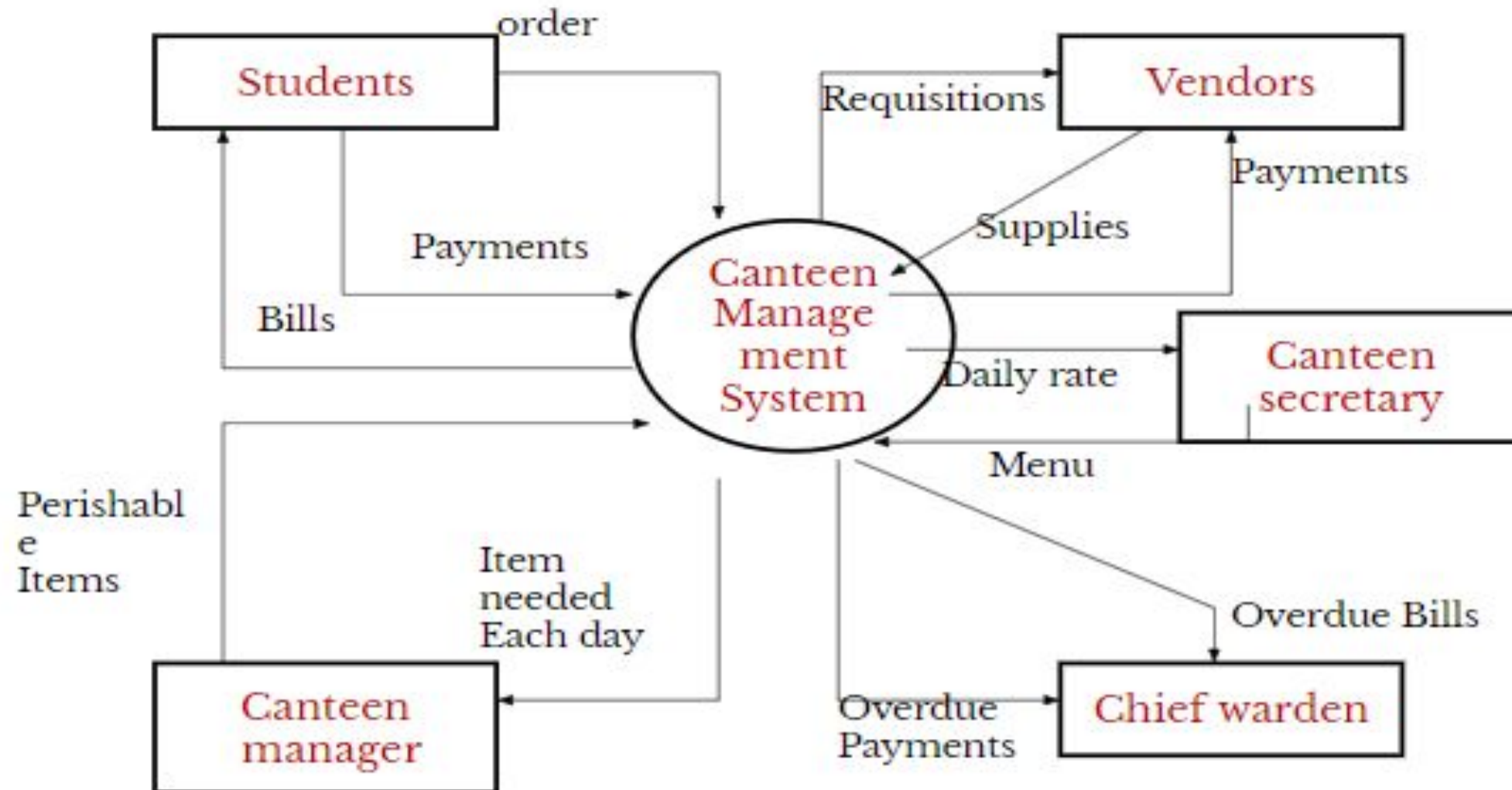
Level 2 DFD

College Management System – Level 2

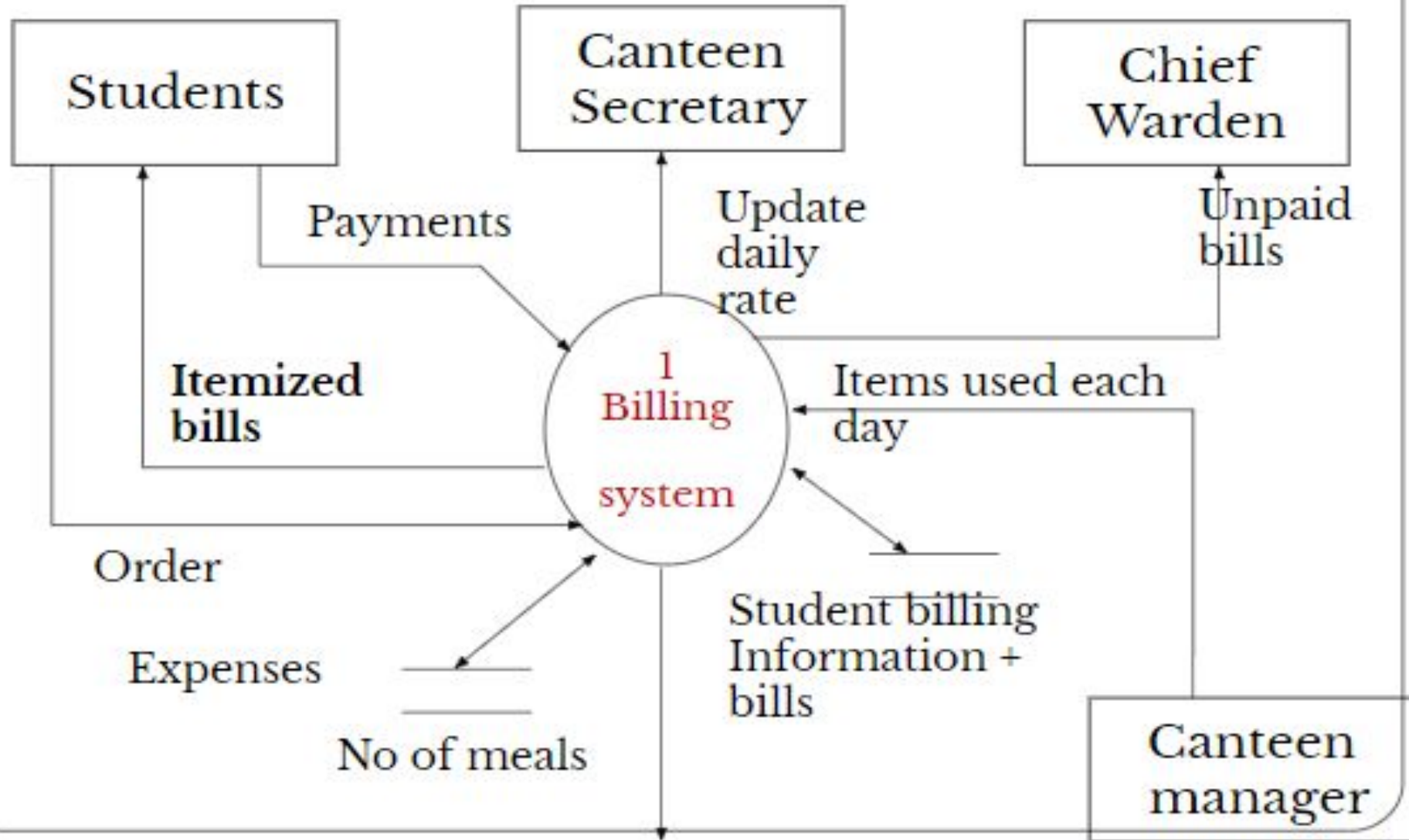
DFD Diagram



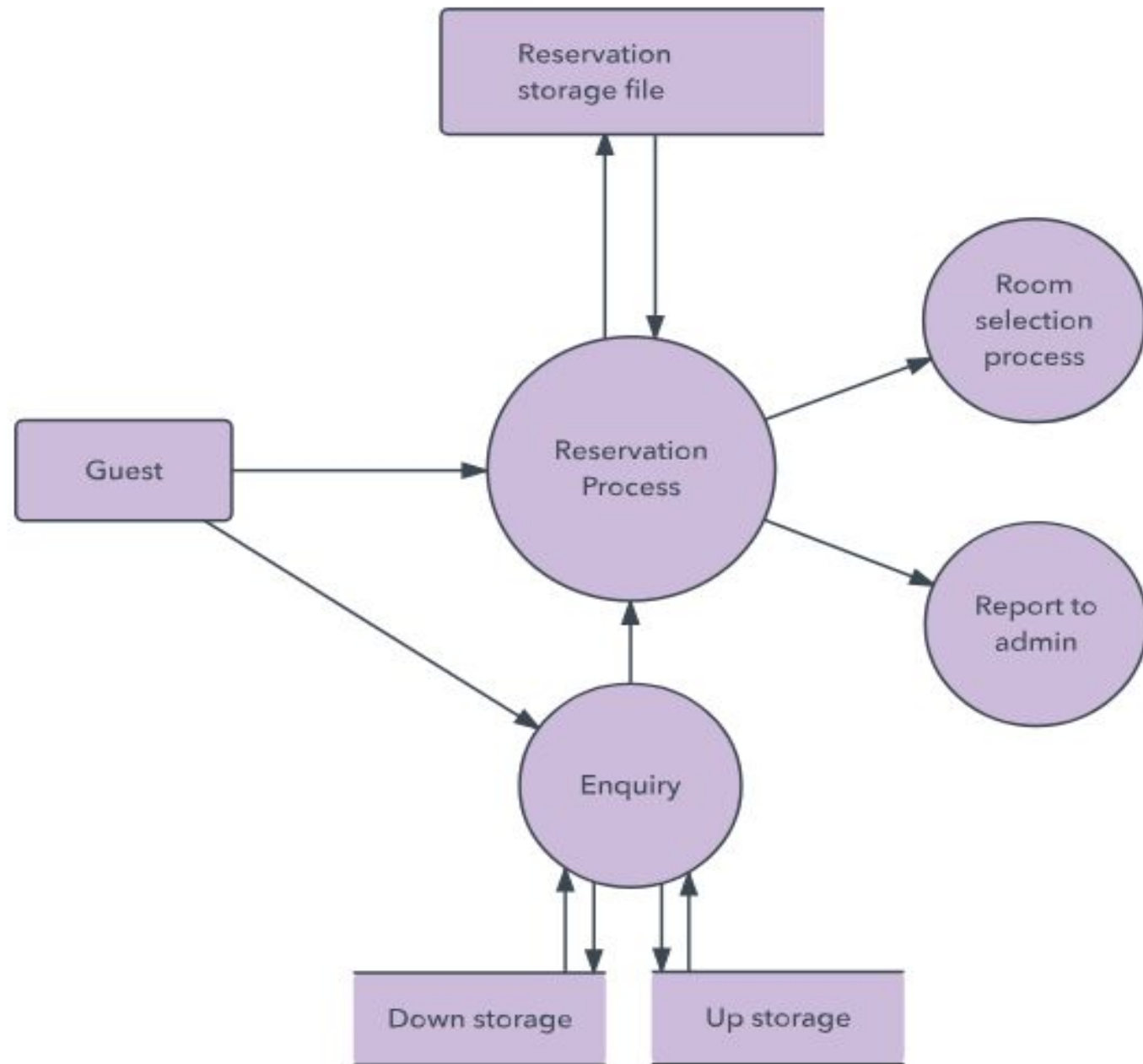
CONTEXT DIAGRAM OF MESS MANAGEMENT SYSTEM

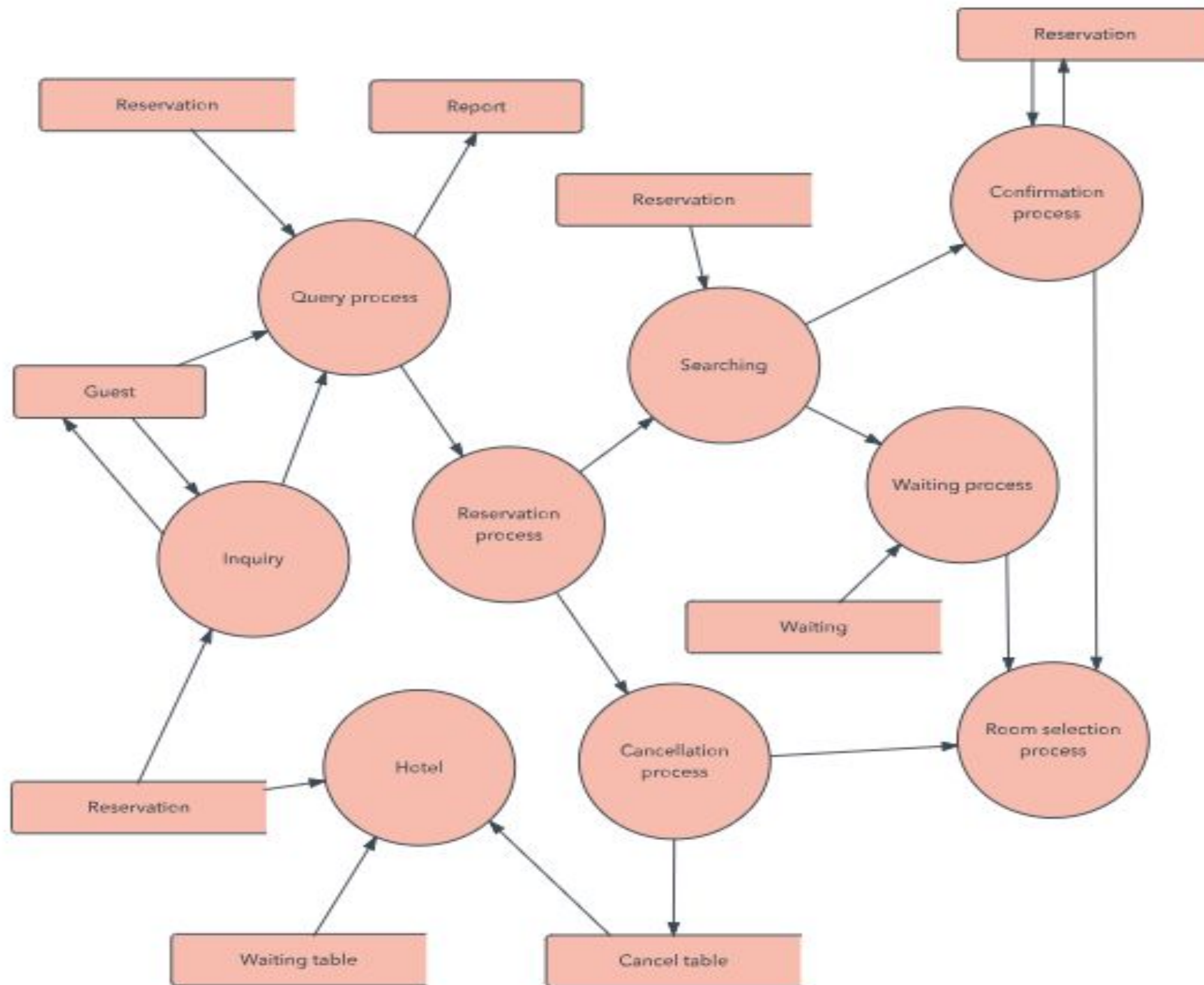


EXPANDED DFD FOR CANTEEN MANAGEMENT SYSTEM







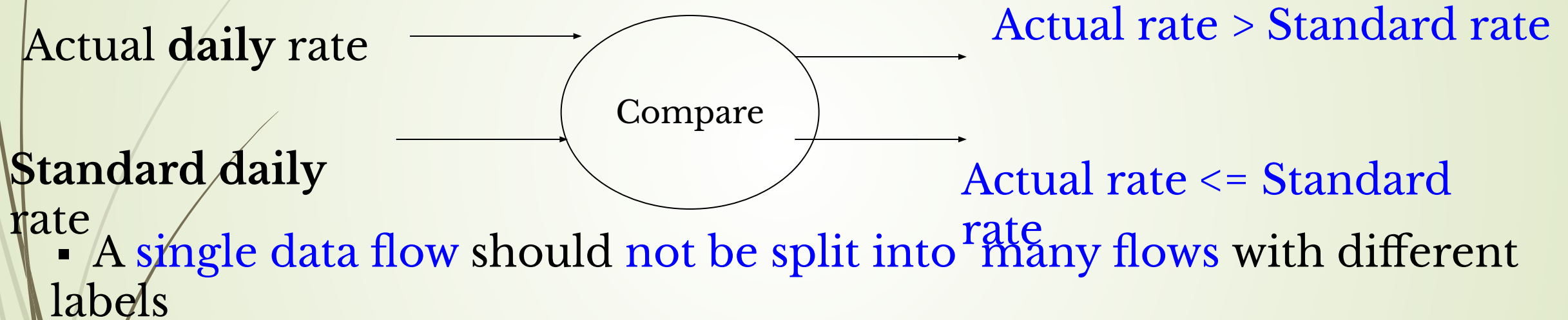


LEVELLING RULES

- If process p is expanded, the process at the next level are labeled as $p.1, p.2$ etc.
- All data flow entering or leaving p must also enter or leave its expanded version.
- **Expanded DFD** may have data stores
- **No external entity** can appear in expanded DFD
- **Keep the number** of processes at each **level less than 7**.

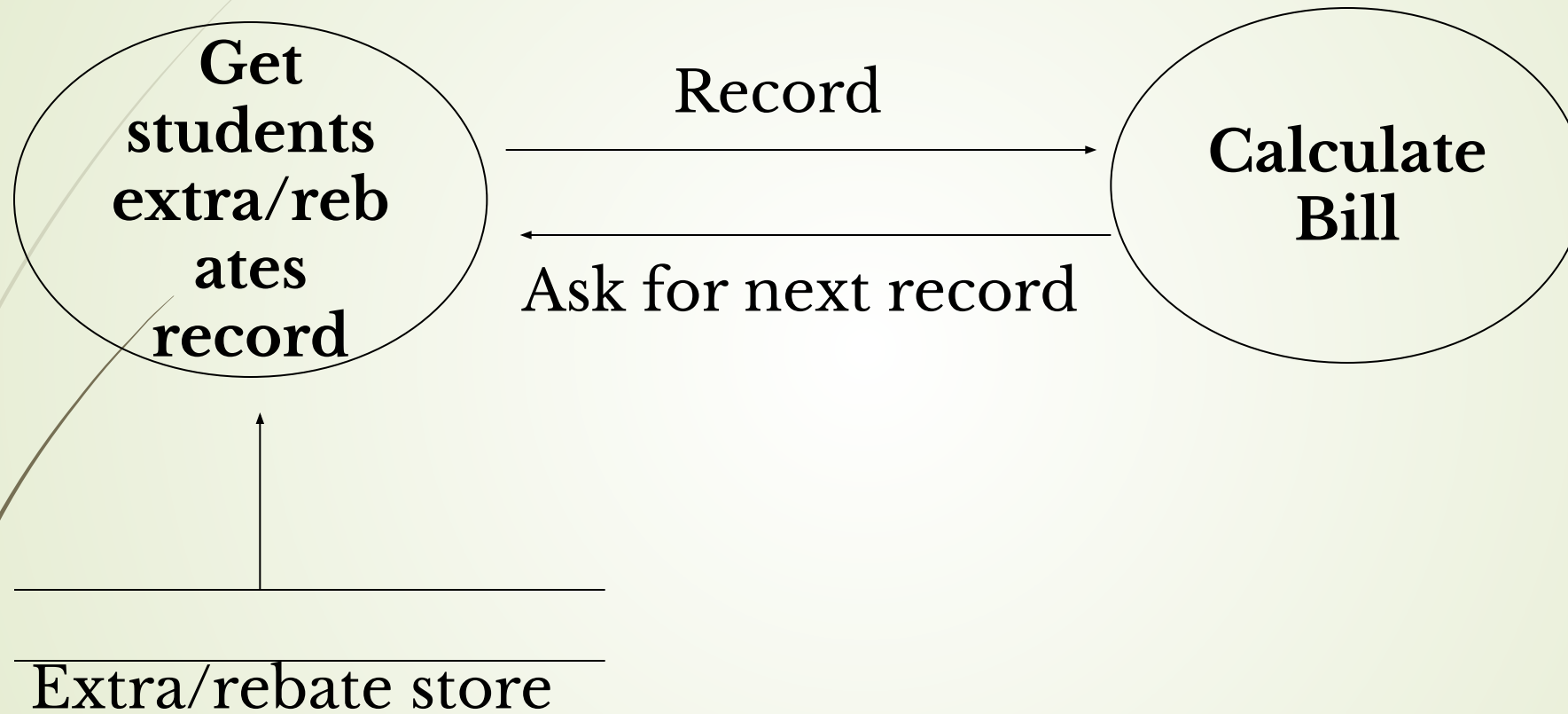
ILLEGAL CONSTRUCTS IN DFD

- **No loops** are allowed in DFD
- **A process cannot be a pure decision**



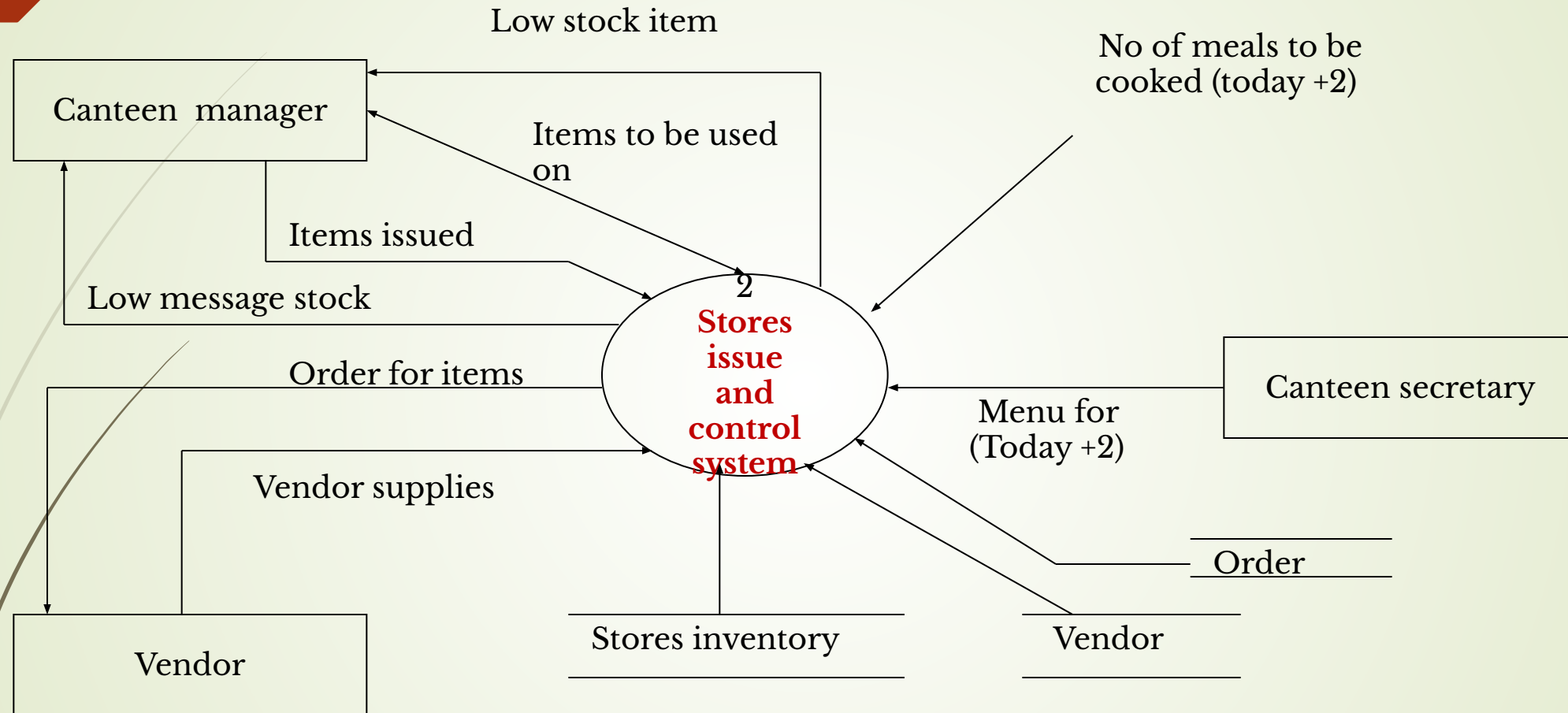
- A **single data flow** should **not be split into many flows** with different labels
- No data flow allowed between data stores

ILLEGAL CONSTRUCTS IN DFD



- **Not correct** as loop is formed

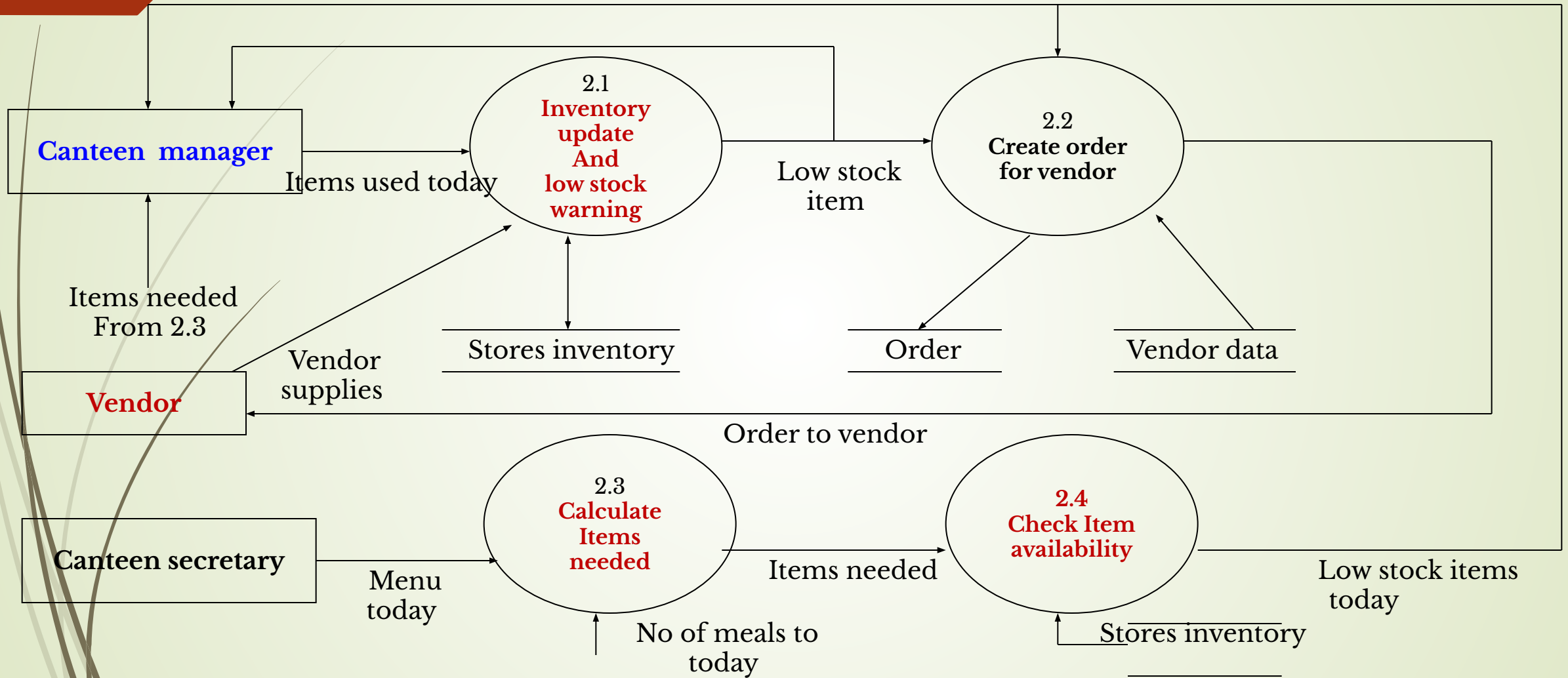
LEVELLING EXAMPLES



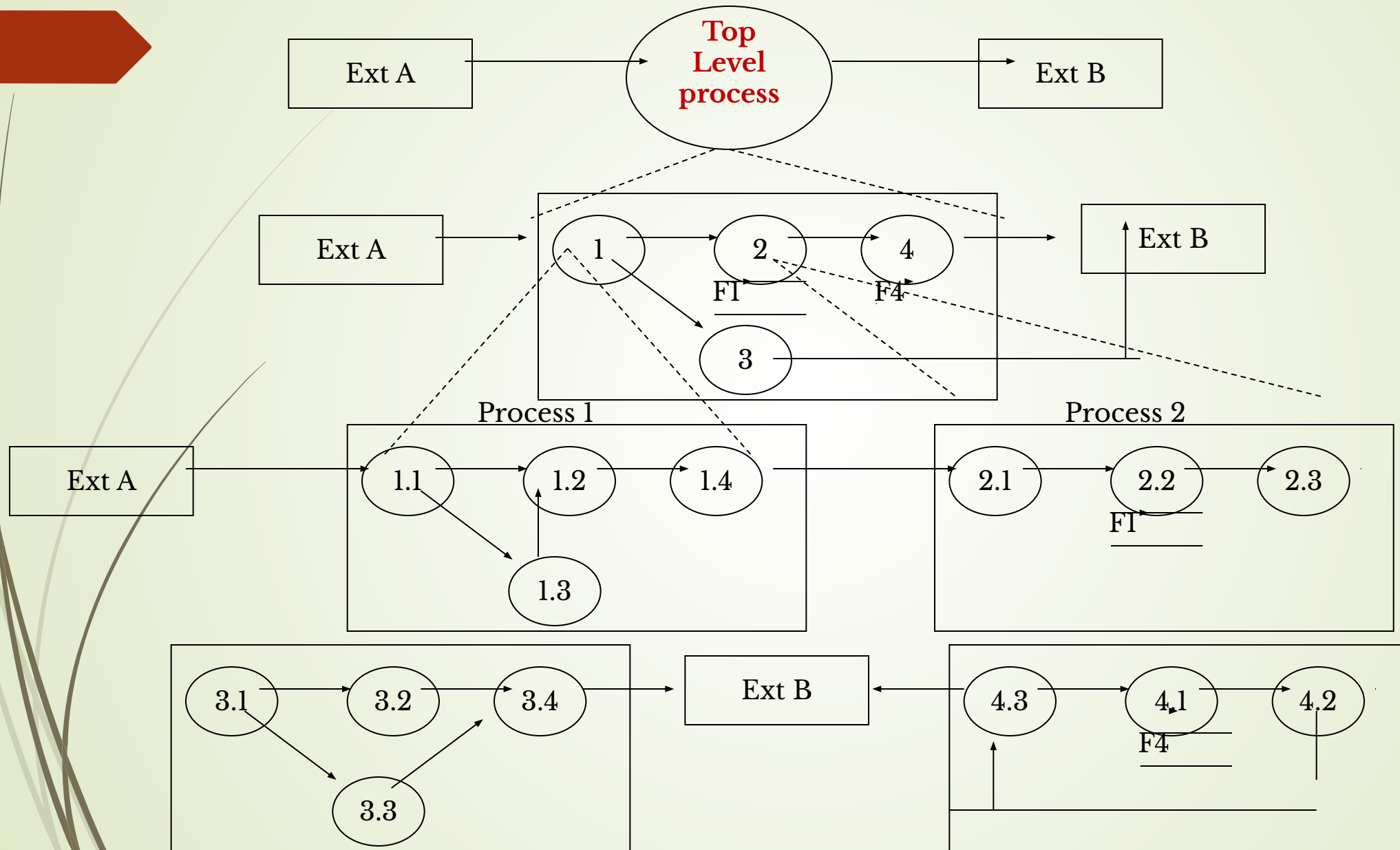
Stores issue control system process

```
graph TD
    CM[Canteen manager]
    V[Vendor]
    CS[Canteen secretary]
    U21((2.1 Inventory update And low stock warning))
    U22((2.2 Create order for vendor))
    U23((2.3 Calculate Items needed))
    U24((2.4 Check Item availability))
    SI1[Stores inventory]
    O[Order]
    VD[Vendor data]

    CM --> U21
    U21 --> CM
    U21 --> U22
    U22 --> CM
    U22 --> O
    U22 --> VD
    CS --> U23
    U23 --> U24
    U24 --> U22
    U24 --> SI2[Stores inventory]
    SI1 <--> U21
    V --> U21
    U21 --> V
```



LEVELLING EXAMPLES



LOGICAL AND PHYSICAL DFD

- DFD'S considered so far are called **logical DFDs**
- A **physical DFD** is similar to a **document flow diagram**.
- It specifies **who does the operations** specified by the logical DFD
- Physical DFD may **depict physical** movements of the goods
- Physical DFDs can be **drawn during fact gathering** phase of a life cycle

II. Behavioral Diagrams

It Capture dynamic aspects or behavior of the system.

Behavior diagrams include:

- Use case diagram
- Sequence diagram

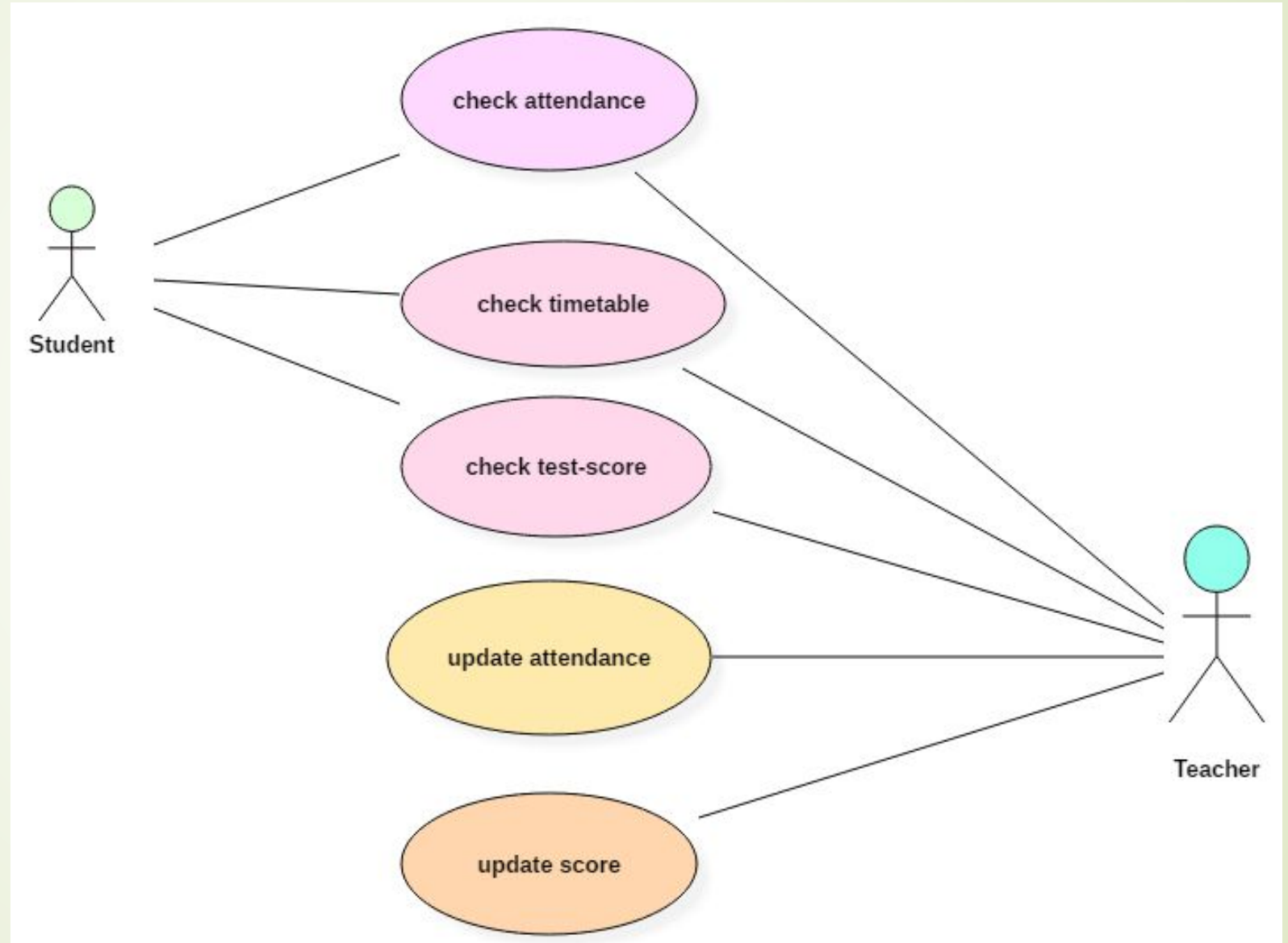
. Use Cases diagram

- ? A use case describes how a user uses a system to accomplish a particular goal.
- ? The actor can be a human, an external system, or time.
- ? It would help us to understand the role of various actors in our project. It helps to organize and plan out things.
- ? Use case diagrams specify the events of a system and their flows. But use case diagram never describes how they are implemented.

5. Use Case Diagram - Example

Student
management
system

Note: It is not necessary that each actor should interact with all the use cases, but it can happen.

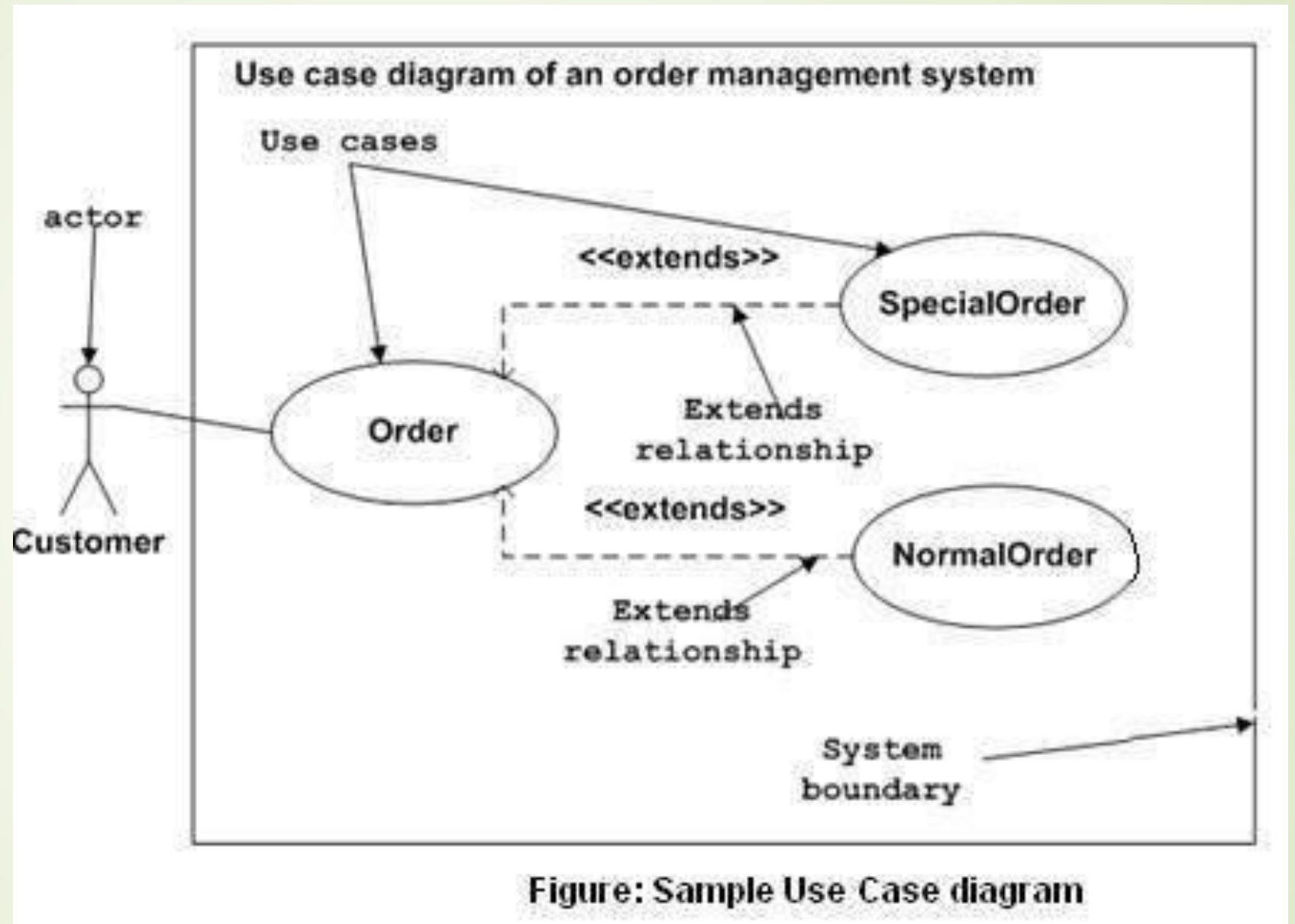


5. Use Case Diagram - Purpose

Use case diagrams are typically develop in early stage of development and people often apply use case modeling for the following purposes:

- Specify the context of a system
- Capture the requirements of a system
- Validate a systems architecture
- Drive implementation and generate test cases
- Developed by analysts together with domain experts

Sample Use Case Diagram



. Use Case Diagram - Notations

1. Actor: The actor can be a human or other external system i.e., someone who interact with use case.



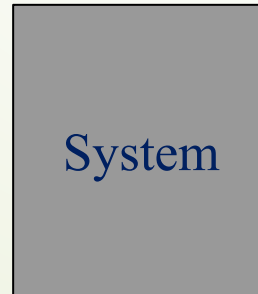
2. Use case: Each Actor must be linked to a use case, while some use cases may not be linked to actors.



Use Case Diagram - Notations

3. Communication link: The participation of an actor in a use case is shown by connecting an actor to a use case by a solid link.

4. Boundary of system: The system boundary is potentially the entire system as defined in the requirements document.



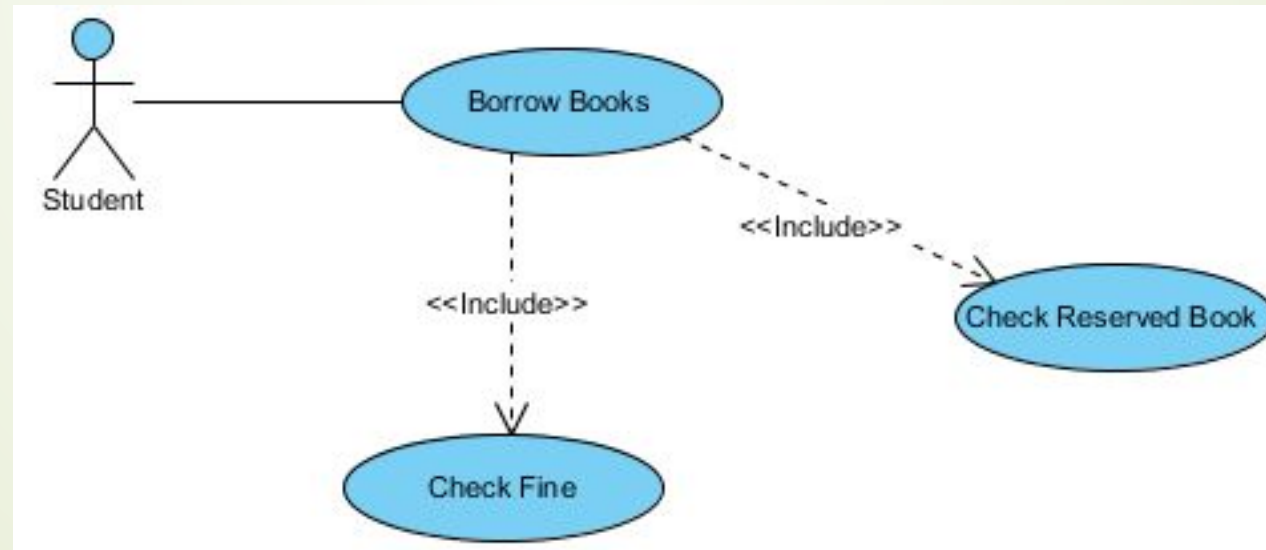
USE CASE DIAGRAM - Relationships

? **Association Link:** A Use Case diagram illustrates a set of use cases for a system, i.e. the actors and the relationships between the actors and use cases.



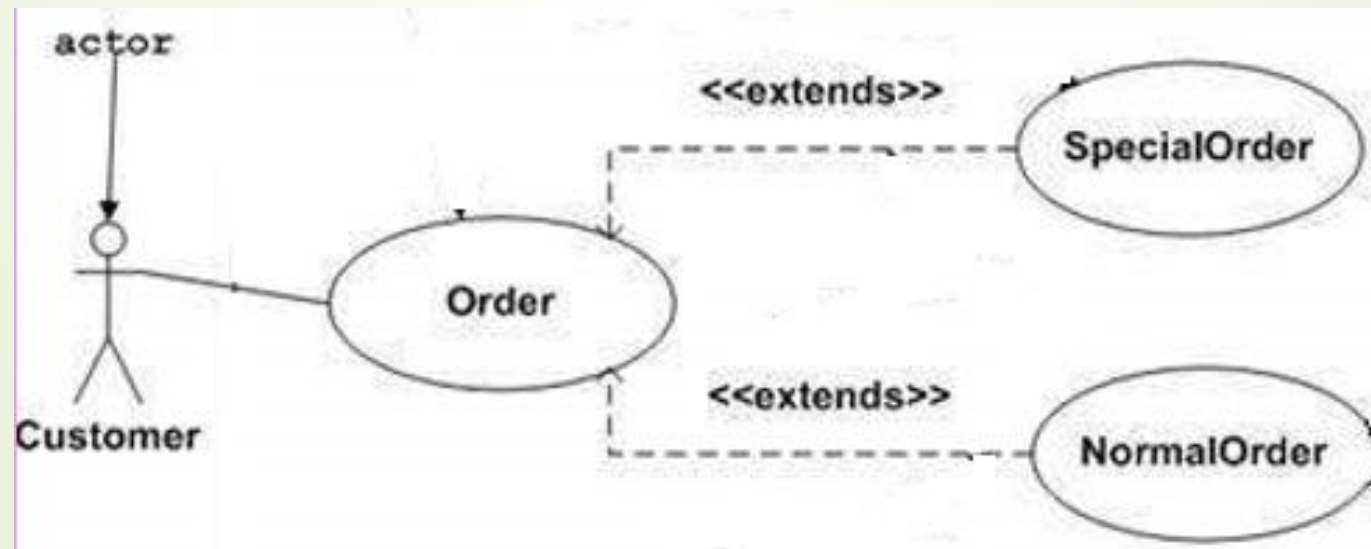
. Use Case Diagram - Relationships

? Include Relationship: The include relationship adds additional functionality not specified in the base use case. It is used to include common behavior from an included use case into a base use case in order to support the reuse of common behavior.



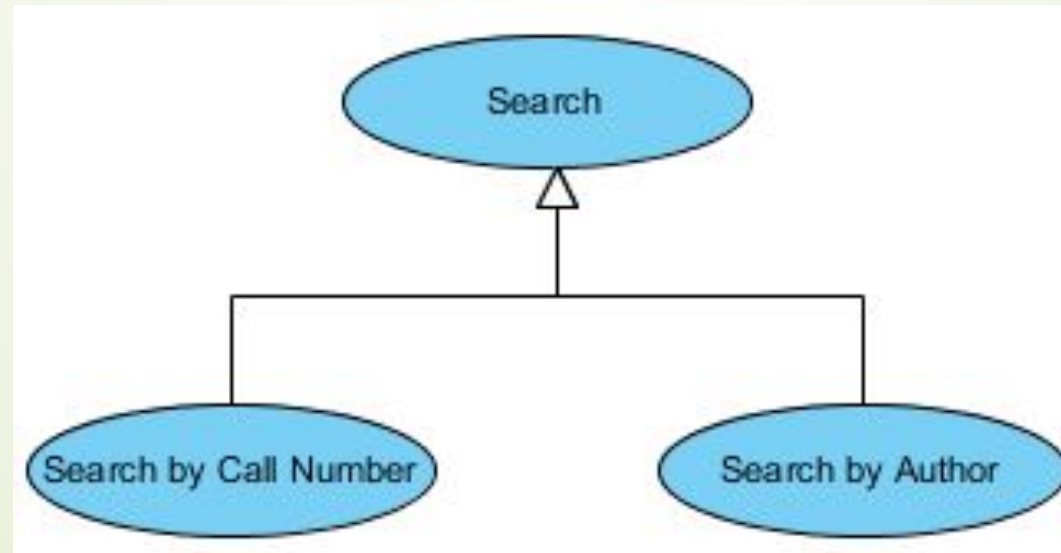
Use Case Diagram - Relationships

? Extend Relationship: The extend relationships are important because they show optional functionality or system behavior. It is used to include optional behavior from an extending use case in an extended use case.



Use Case Diagram - Relationships

? **Generalization Relationship:** It means that a child use case inherits the behavior and meaning of the parent use case. The child may add or override the behavior of the parent. The figure below provides a use case example by showing two generalization connectors that connect between the three use cases.



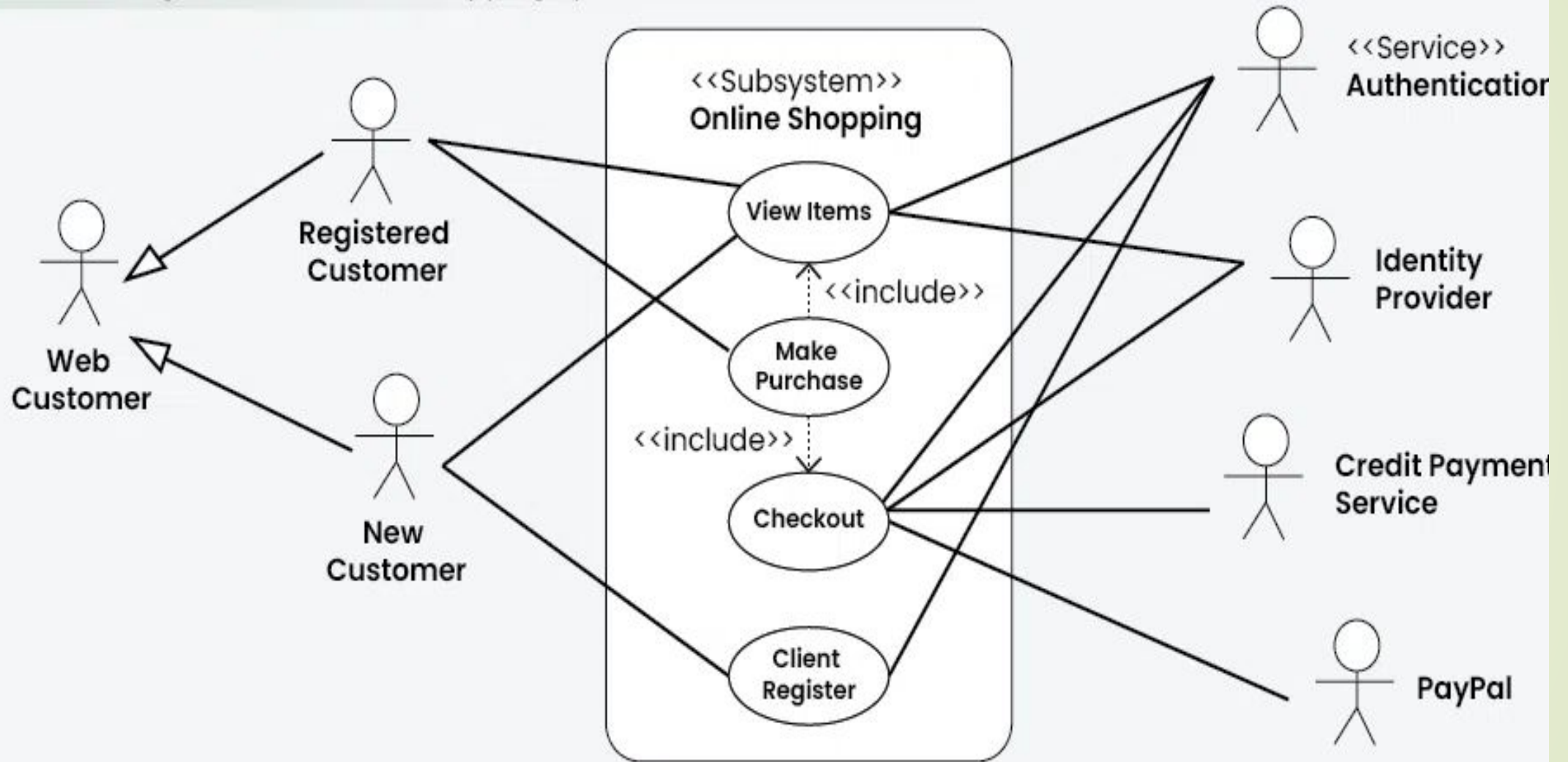
How to draw a Use Case Diagram?

1. Identify the Actors (role of users) of the system.
2. For each category of users, identify all roles played by the users relevant to the system.
3. Identify what are the users required the system to be performed to achieve these goals.
4. Create use cases for every goal.
5. Structure the use cases.
6. Prioritize, review, estimate and validate the users.

5. Use Case Diagram - Benefits

- ? It is a powerful technique for the elicitation and documentation of black-box functional requirements.
- ? It provides an excellent way for communicating with customers and users as they are written in natural language.
- ? It can help manage the complexity of large projects by partitioning the problem into major user features (i.e., use cases) and by specifying applications from the users' perspective.

Use Case diagram of an Online Shopping System



Sequence diagram

- ? Sequence Diagrams are interaction diagrams that detail how operations are carried out.
- ? It captures high-level interactions between user of the system and the system, between the system and other systems, or between subsystems.
- ? Sequence Diagrams are time focus and they show the order of the interaction visually by using the vertical axis of the diagram to represent time what messages are sent and when.

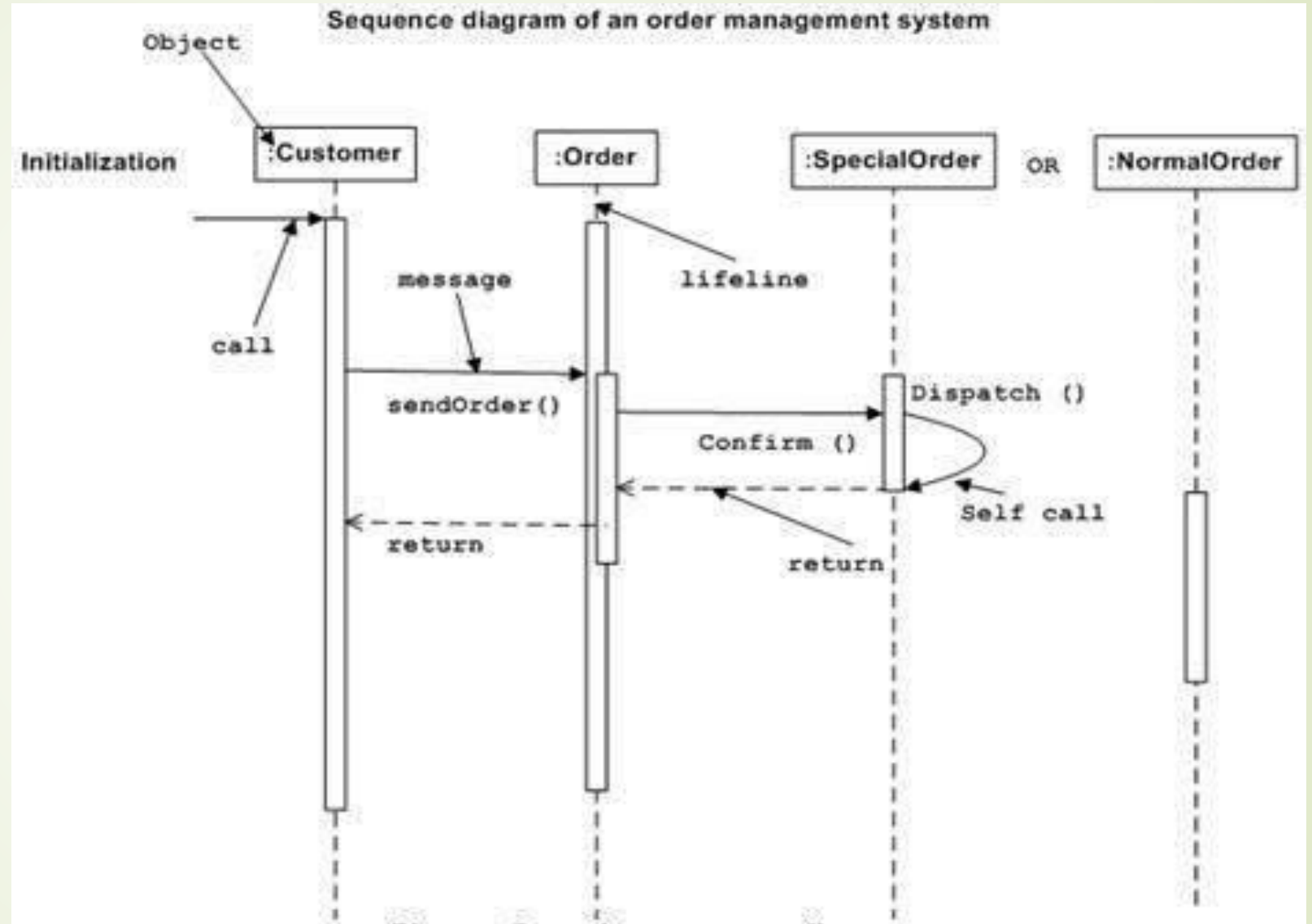
Sequence diagram - Elements

Sequence Diagrams show elements as they interact over time and they are organized according to object (horizontally) and time (vertically):

- ? **Object Dimension:** The horizontal axis shows the elements that are involved in the interaction.
- ? **Time Dimension:** The vertical axis represents time proceedings (or progressing) down the page.

Sequence diagram - Example

The sequence diagram has four objects: (Customer, Order, SpecialOrder and NormalOrder).



Requirement Modelling

- ? Requirements modeling is the process used in software development projects where requirements and solutions constantly evolve through collaborative efforts and teamwork.
- ? To achieve fast, consistent and continuous delivery of your software, requirements modeling is critical.

Requirement Model

It is the process of identifying the requirements this software solution must meet in order to be successful. The requirements modeling action results in one or more of the following types of models:

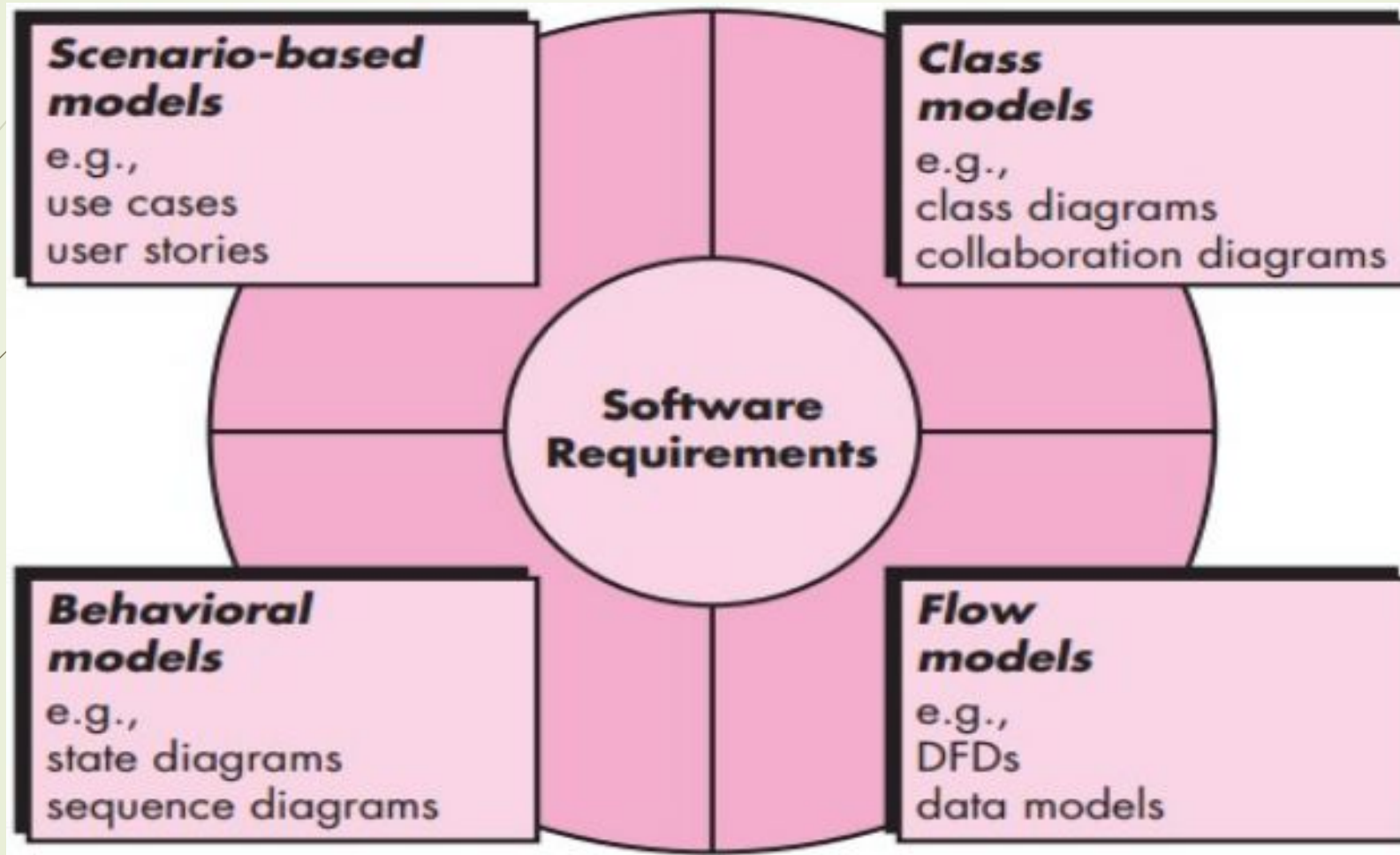
1. **Scenario-based models** of requirements from the point of view of various system “actors”
2. **Class-based models** that represent object-oriented classes (attributes and operations) and the manner in which classes collaborate to achieve system requirements
3. **Behavioral models** that depict how the software behaves as a consequence of external “events”

Requirement Model- Objectives

The requirements model must achieve three primary objectives:

- (1) to describe what the customer requires,
- (2) to establish a basis for the creation of a software design,
and
- (3) to define a set of requirements that can be validated once the software is built

Requirement Model



1. Scenario-based model

- ? Requirements modeling typically begins with scenario-based modeling because it identifies the possible use cases for the system and produces the use case diagram, to which all the other stages of requirements modeling refer.
- ? It represents the system user point of view.
- ? Scenario based elements are use case diagram, user stories.

2. class-based model

- ? Class-based modeling represents the objects that the system will manipulate, the operations that will be applied to the objects to effect the manipulation, relationships (some hierarchical) between the objects, and the collaborations that occur between the classes that are defined.
- ? Class based elements are the class diagram, collaboration diagram.

3. Behavioral models

- ? It represent state of the system and how it is changed by the external events.
- ? The behavioral elements are sequenced diagram, state diagram.